



MY ARENA
MY SPORT ZONE

FINAL PROJECT REPORT

FOR THE MY ARENA APPLICATION

ABAIR NAIMI Fatima Azahara
BEN AYED Ismaël
BEN NACEF Wassim
BENALI Syrine

FINAL PROJECT REPORT.....	1
FOR THE MY ARENA APPLICATION.....	1
Introduction.....	5
I. Use Case analysis:.....	6
A. Overview of actors:.....	6
B. Key Use cases:.....	6
C. Use Case Analysis Management Report:.....	7
1. Project Overview.....	7
2. Main Actors Identified.....	7
3. Use Cases Identified.....	7
4. Key Decisions and Changes Made.....	7
4.1 Tournament Reservations.....	7
4.2 Payment System Replaced by Cart Management.....	8
4.3 Promotion Management.....	8
4.4 User Role Management.....	8
5. Work Distribution.....	8
6. Documentation Structure.....	8
7. Tools and Deliverables.....	9
7.1 UML Diagrams.....	9
7.2 User Interface Mockups.....	9
8. Next Steps and Pending Items.....	9
9. Conclusion.....	9
II. Domain Analysis:.....	10
A. Domain model overview:.....	10
B. Domain Class diagram:.....	10
C. Domain Analysis report:.....	11
1. Project Overview.....	11
2. Objectives of the Session.....	11
3. Process Followed During the Session.....	11
3.1. Preparing the Environment.....	11
3.2. Brainstorming Based on Use Cases.....	11
3.3. Reviewing Course Material.....	12
3.4. Structuring the Model Into Packages.....	12
3.5. Building the UML Class Diagram.....	13
3.6. Final Model Validation.....	13
4. Final Domain/System Model Overview.....	13
5. Key Decisions and Changes Made.....	14
5.1. Consolidation of Redundant Entities.....	14
5.2. Differentiation Between Product Sale and Equipment Rental.....	14
5.3. Enhanced Notification Model.....	14
5.4. Explicit History and Tracking Entities.....	14

5.5. Status-Driven Domain Logic.....	14
6. Work Distribution.....	14
7. Tools and Deliverables.....	15
7.1. UML Modeling Tool.....	15
7.2. Deliverables Produced.....	15
8. Next Steps and Pending Items.....	15
9. Conclusion.....	16
III. System design and architecture:.....	16
A. Architectural pattern:.....	16
B. Prototype Diagrams:.....	17
1. Class Diagram:.....	17
2. Sequence Diagram.....	18
C. Prototype Management Report:.....	18
1. Project Overview.....	18
2. Objectives of the Session.....	19
3. Process Followed during the Session.....	19
4. Work Distribution.....	21
5. Tools and Deliverables.....	21
6. Next Steps and Pending Items.....	21
IV. First Sprint:.....	22
A. Class Diagrams:.....	22
1. Authentication Management:.....	22
2. Reservation Management:.....	23
3. Terrain Management:.....	24
4. Subscription Management:.....	25
B. Sequence Diagrams:.....	29
1. Authentication Management:.....	29
2. Reservation Management:.....	32
3. Terrain Management:.....	33
4. Subscription Management:.....	34
C. Sprint 1 Management Report:.....	34
1. Project Overview.....	34
2. Objectives of the Session.....	35
3. Process Followed during the Session.....	35
4. Work Distribution.....	36
5. Tools and Deliverables.....	36
5.1. UML Modeling Tool.....	36
5.2. Deliverables Produced.....	36
6. Process Metrics and Quantitative Overview.....	37
7. Problems Encountered and Lessons Learned.....	37
8. Next Steps and Pending Items.....	37

9. Conclusion.....	38
V. Second Sprint:.....	39
A. Class Diagrams:.....	39
1. Cart and Order Management:.....	39
2. Equipment Management:.....	40
3. Notification Management:.....	41
4. Tournament Management:.....	43
B. Sequence Diagrams.....	44
1. Cart and Order Management.....	44
2. Notification Management:.....	46
3. Equipment Management:.....	47
C. Second Sprint Management Report:.....	49
1. Project Overview.....	49
2. Objectives of the Session.....	49
3. Process Followed during the Session.....	49
4. Work Distribution.....	50
5. Tools and Deliverables.....	50
5.1. UML Modeling Tool.....	50
5.2. Deliverables Produced.....	51
6. Process Metrics and Quantitative Overview.....	51
7. Problems Encountered and Lessons Learned.....	51
8. Next Steps and Pending Items.....	52
9. Conclusion.....	52

Introduction

My Arena is an intermediary platform connecting three main user groups: clients, tournament organizers, and sports facility owners. The platform simplifies communication, helps organizers find suitable venues, and allows field owners to showcase their available facilities and equipment.

This final report serves as the comprehensive documentation for the project, covering the entire software development lifecycle from functional analysis to detailed design and implementation.

I. Use Case analysis:

A. Overview of actors:

- ❖ **Client:** Books facilities, registers for tournaments, rents/buys equipment, and manages subscriptions.
- ❖ **Owner:** Manages facilities and equipment, validates payments, and applies promotions.
- ❖ **Organizer:** Creates and manages tournaments.
- ❖ **Administrator:** Manages users, roles, subscription plans, and validates payments.
- ❖ **System:** Sends automated notifications to all users.

B. Key Use cases:

The system functionality is divided into 8 main families:

- ❖ **User Management:** Registration, Login, Profile Management. Note: Users register as "Client" by default; Role changes to Owner/Organizer must be requested from an Admin.
- ❖ **Terrain/Installation Management:** CRUD operations for facilities, availability definitions, and search functionality.
- ❖ **Reservation Management:** Creating, modifying, and canceling reservations for terrains.
- ❖ **Subscription Management:** Managing subscription plans (Admin) and subscribing to plans (Client).
- ❖ **Tournament Management:** Creating tournaments (Organizer), approving them (Admin), and registering for them (Client).
- ❖ **Equipment Rent/Sale:** Managing the equipment catalog (Owner) and purchasing/renting items (Client).
- ❖ **Cart Management:** Handling orders. Note: Automatic payment was replaced by a "Cart & Order Validation" process where Owners manually validate payments.
- ❖ **Notifications:** A central system for users to view and receive updates.

C. Use Case Analysis Management Report:

Date: Friday, November 21, 2024

Duration: 3 hours (+ individual work until Sunday evening)

Team Members: 4 people

Deadline: Sunday, November 24, 2024

1. Project Overview

Our team is developing a Java application that serves as an intermediary platform connecting three main user groups: clients, tournament organizers, and sports facility owners. The platform allows clients to book sports facilities, participate in tournaments, and rent or buy sports equipment. Facility owners can manage their properties, define availability, and sell equipment. Tournament organizers can create and manage sporting events.

2. Main Actors Identified

Client: Books facilities, registers for tournaments, rents/buys equipment, manages subscriptions

Owner: Manages facilities and equipment, validates payments, applies promotions

Organizer: Creates and manages tournaments

Administrator: Manages users, roles, subscription plans, and validates payments

System: Sends automated notifications to all users

3. Use Cases Identified

During this session, we validated and finalized **8 main use case families**:

- **User Management** - Register, Login, Manage Profile, Maintain Users (Admin)
- **Terrain/Installation Management** - CRUD operations, define availability, search facilities
- **Reservation Management** - Create, modify, cancel reservations, view history
- **Subscription Management** - View plans, subscribe, manage plans (Admin)
- **Tournament Management** - Register, create, modify, delete tournaments
- **Equipment Rent/Sale** - Maintain equipment catalog, browse, rent, and buy products
- **Cart Management** - Make orders, validate payment, apply promotions
- **Notifications** - View notification center, receive system notifications

4. Key Decisions and Changes Made

4.1 Tournament Reservations

Initially, we separated terrain reservations for tournaments from regular client reservations. After discussion, we decided they follow the same process and merged them into a single use case to avoid redundancy.

4.2 Payment System Replaced by Cart Management

We replaced the original "Payment" use case with "Cart Management," which simulates the payment process. Instead of automatic payment, owners must manually validate orders after clients submit them. This approach simplifies development while maintaining control over transactions.

4.3 Promotion Management

We discussed how to handle promotions and discounts. The final decision was to treat promotions as product attributes that owners can define directly. This gives owners flexibility to set discounts on specific equipment or services.

4.4 User Role Management

One major debate concerned how to assign user roles. Initially, we planned for administrators to register all users and assign their roles. However, this would create too much work for administrators. Our final solution: when users register, they are assigned the "Client" role by default. If they want to become an Owner or Organizer, they must contact the administrator outside the application to request a role change.

5. Work Distribution

The 8 use case families were distributed equally among the 4 team members:

- **Member 1:** User Management (1) + Cart Management (7)
- **Member 2:** Terrain Management (2) + Reservation Management (3)
- **Member 3:** Subscription Management (4) + Notifications (8)
- **Member 4:** Tournament Management (5) + Equipment Rent/Sale (6)

Each member was assigned complementary use cases to ensure better understanding of connected functionalities and to facilitate detailed specification work.

6. Documentation Structure

For each use case, we followed this standardized template:

- **Brief Description**
- **Flow of Events**
 - Basic Flow
 - Alternative Flows
- **Special Requirements**
- **Pre-Conditions**
- **Post-Conditions**
- **Extension Points**

This structure ensures comprehensive documentation and helps developers understand the complete behavior of each functionality.

7. Tools and Deliverables

7.1 UML Diagrams

- **Tool used:** UMLet.ino (<https://www.umletino.com/umletino.html>)
- **Diagrams created:**
 - Individual use case diagrams for each of the 8 families
 - One global use case diagram showing the complete system
 - Activity diagrams showing process flows (e.g., cart management workflow)

7.2 User Interface Mockups

- **Tool used:** Figma
- **Process:** After discussing interface requirements together, we created mockups for main screens including catalog browsing, reservation forms, cart management, and user profiles.

8. Next Steps and Pending Items

After this session, each team member must finalize their assigned use cases individually before Sunday evening.

This includes:

- Completing detailed specifications for both use cases
- Finishing UML diagrams
- Verifying consistency with other team members' work
- The complete documentation must be ready before the deadline to allow time for final verification and submission.

9. Conclusion

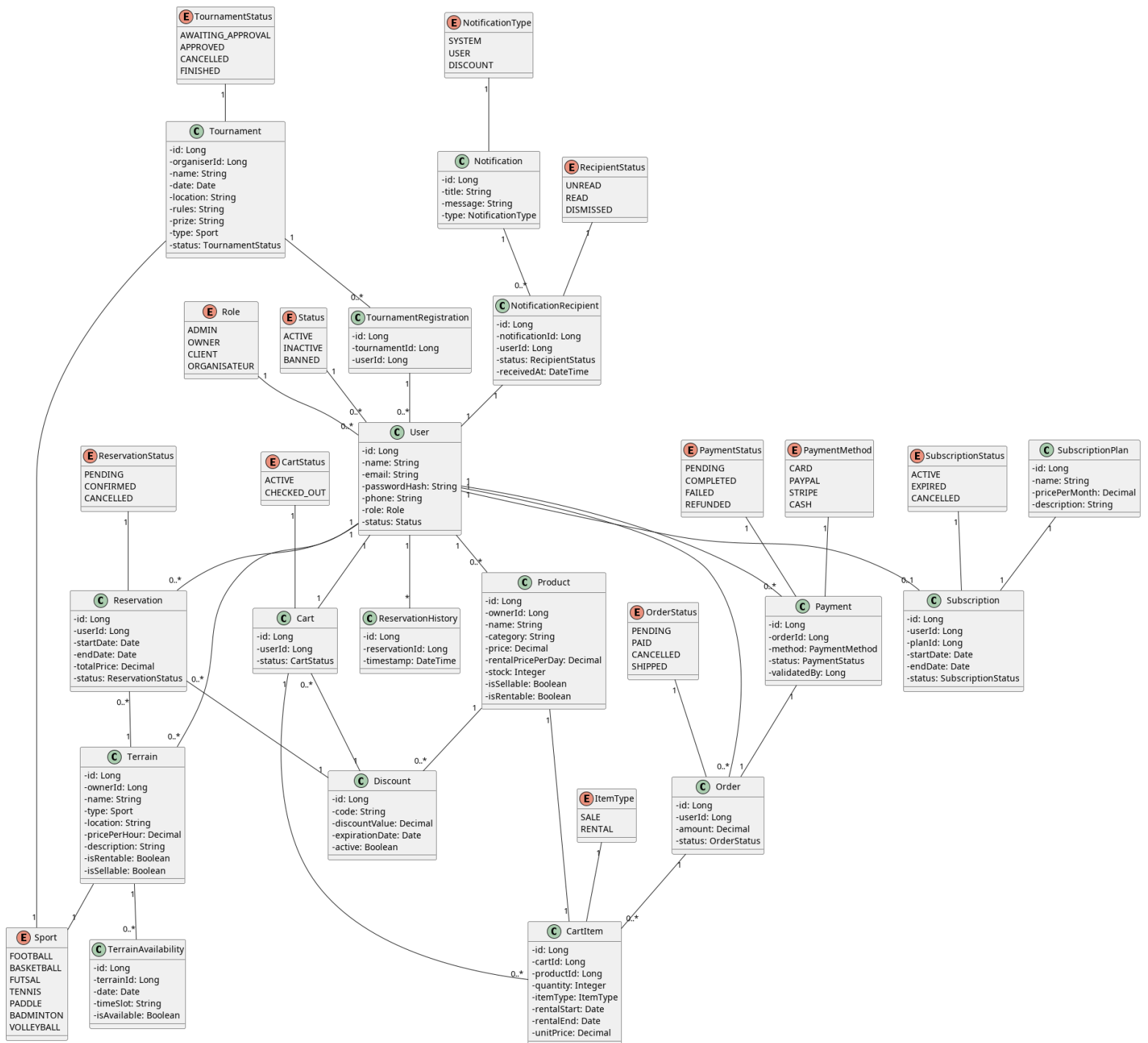
This work session was productive and allowed us to validate our use case model after making important adjustments. The key decisions regarding tournament reservations, payment validation, promotions, and role management will significantly impact the application's architecture. The clear work distribution ensures that each team member can work efficiently while maintaining overall consistency in the project documentation.

II. Domain Analysis:

A. Domain model overview:

The domain analysis established the core entities and relationships of the system. We utilized PlantUML to generate the model and ensure compatibility with Java code generation.

B. Domain Class diagram:



C. Domain Analysis report:

Date: Friday, November 28, 2025

Duration: 3 hours

Team Members: 4 people

Deadline: Sunday, November 30, 2025

Manager: BEN NACEF Wassim

1. Project Overview

Our team is developing a Java application that serves as an intermediary platform connecting three main user groups: clients, tournament organizers, and sports facility owners. The platform allows clients to book sports facilities, participate in tournaments, and rent or buy sports equipment. Facility owners can manage their properties, define availability, and sell equipment. Tournament organizers can create and manage sporting events.

2. Objectives of the Session

According to the phase instructions, the team was required to:

1. **Create the domain/system model** using a UML tool capable of separating the model from diagrams and generating Java code (PlantUML in our case).
2. **Build the complete class diagram** covering all entities, enums, and relationships extracted from the use cases.
3. Prepare a written document including the full class diagram and explanations.

All members collaborated since this phase was defined as **group work**.

3. Process Followed During the Session

3.1. Preparing the Environment

Each team member installed and tested the required tools (PlantUML, project documents, guidelines). This ensured everyone could work on the same model structure and export consistent diagrams/code.

3.2. Brainstorming Based on Use Cases

We systematically reviewed all use case families validated in the previous phase (Authentication, Terrain Management, Reservation, Subscription, Tournament, Equipment Rent/Sale, Cart, Notifications).

For each use case, we extracted:

- Candidate **classes**
- Potential **attributes**
- Required **associations**
- Related **enumerations**
- Domain rules implied by flows or alternative flows

This allowed us to gradually build an exhaustive list of domain concepts.

3.3. Reviewing Course Material

We referred to class extraction guidelines from the lectures to avoid duplicating responsibilities or creating procedural-style classes.

This helped distinguish:

- **Entities** (User, Reservation, Order, Payment...)
- **Value types** (DateTime, price fields, IDs...)
- **Enumerations** (Statuses, Payment Methods...)
- **Associations** (User ↔ Reservation, Tournament ↔ Registration...)

3.4. Structuring the Model Into Packages

We grouped the emerging classes into domain packages such as:

- User & Roles
- Tournaments

- Reservations
- Products & Equipment
- Orders & Payments
- Notifications
- Subscriptions
- Terrains & Availability

3.5. Building the UML Class Diagram

We then moved to PlantUML to construct the final domain model:

- All classes were added with attributes and types.
- Multiplicities were carefully defined (1..*, 0..1, etc.).
- Enums were placed near their related entities.
- Bi-directional relationships were resolved or simplified when unnecessary.

This step required several iterations to harmonize naming conventions and remove redundancies.

3.6. Final Model Validation

We validated the final diagram by verifying that:

- every use case from the previous phase is supported by the model,
- no actor interaction is missing a corresponding entity,
- no entity is unnecessary or unused,
- relationships are consistent with the business logic,
- enumerations correctly represent domain constraints (statuses, types...).

4. Final Domain/System Model Overview

The final class diagram includes all major classes such as:

- **User, Role, Status, Reservation, Terrain, Tournament, Notification, Product, Cart, Order, Payment, Subscription**, etc.
- All required **enumerations**: TournamentStatus, PaymentStatus, ReservationStatus, SubscriptionStatus, CartStatus...
- Full **associations** with correct multiplicities.

This domain model formalizes the complete static structure of the MyArena system and ensures alignment with future implementation.

5. Key Decisions and Changes Made

5.1. Consolidation of Redundant Entities

Some early candidate classes from brainstorming were merged (e.g., separate TournamentApproval entities were not needed thanks to TournamentStatus).

5.2. Differentiation Between Product Sale and Equipment Rental

We decided to maintain a single **Product** class with an enum **ItemType (SALE / RENTAL)** to avoid duplicating attributes.

5.3. Enhanced Notification Model

We introduced **NotificationRecipient** to support multiple users receiving the same system notification and tracking read/dismissed states.

5.4. Explicit History and Tracking Entities

ReservationHistory and CartItem were added to reflect use case flows and maintain business traceability.

5.5. Status-Driven Domain Logic

Statuses were modeled as enums instead of strings or separate classes to enforce system consistency and simplify code generation.

6. Work Distribution

While brainstorming and validation were collaborative, the construction of the UML model was done cooperatively in real-time through screen sharing and parallel reviews. Tasks were distributed as follows:

- **Wassim:** User, Role, Status, Subscription
- **Syrine:** Terrains, Availability, Reservations
- **Fatima :** Products, Cart, Orders, Payments
- **Ismael :** Tournaments, Registrations, Notifications

Everyone participated in reviewing the final diagram before exporting it.

7. Tools and Deliverables

7.1. UML Modeling Tool

PlantUML

- Supported the creation of the domain model
- Enabled separation of model/diagrams
- Enabled automatic Java code generation

7.2. Deliverables Produced

- Complete UML class diagram (final version integrated in the report)
- Export of the domain model as Java skeleton code
- Written report describing the model and process

8. Next Steps and Pending Items

After completing this phase, the next objectives will be:

- Reviewing the generated Java code and fixing structural issues
- Beginning architectural design (layers, repositories, services...)
- Preparing sequence diagrams or interaction models if required
- Collecting feedback from teacher assistants

9. Conclusion

This session successfully transformed the functional requirements of MyArena into a well-structured, comprehensive domain system model. Through iterative brainstorming, careful use case extraction, and collaborative diagram building, we established a clear and consistent representation of the system's core entities and relationships.

The resulting domain model will play a central role in guiding the future implementation and ensuring architectural consistency across the project.

III. System design and architecture:

A. Architectural pattern:

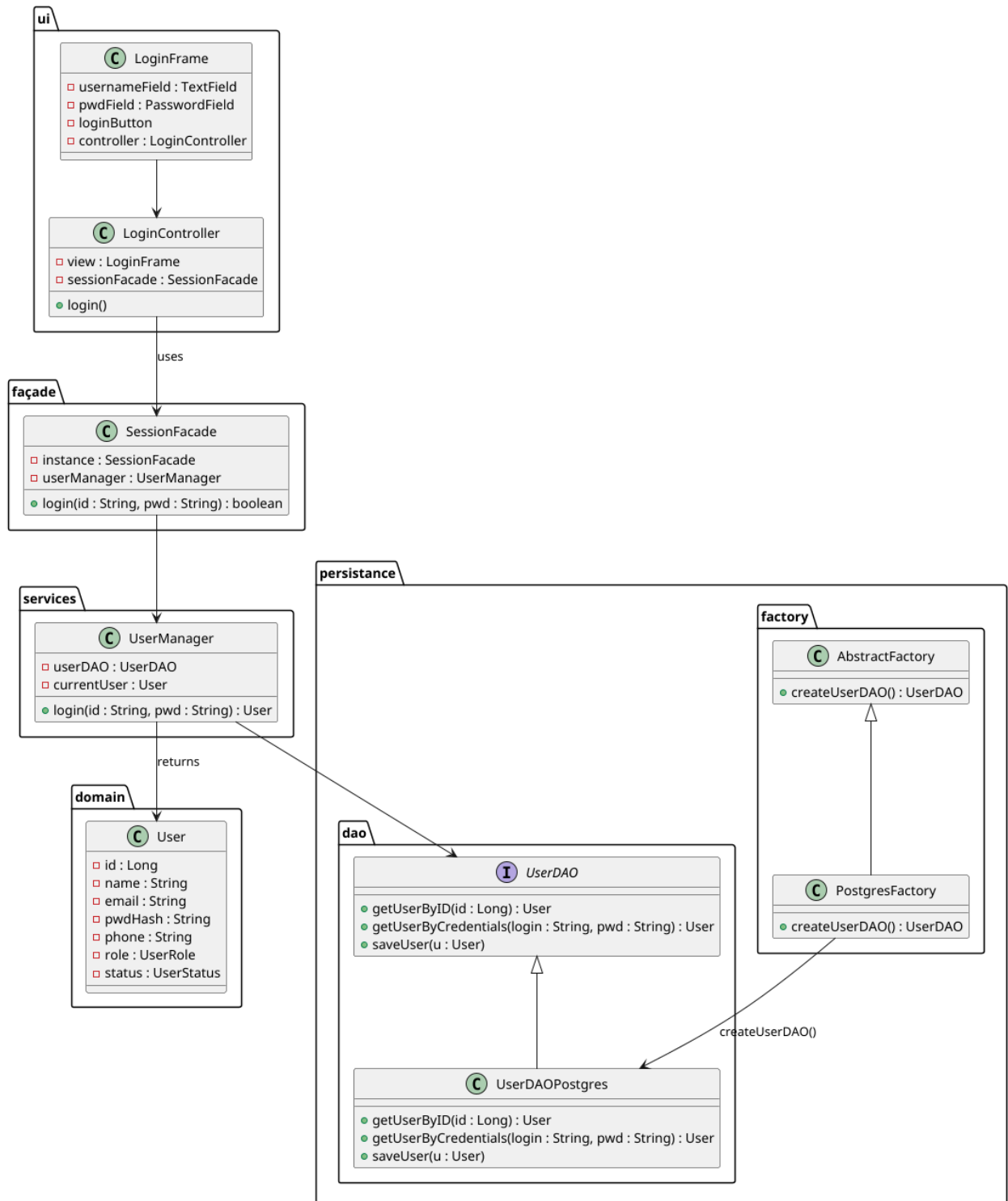
The application follows a Layered Architecture strictly separating concerns to ensure maintainability and scalability:

- ❖ **UI Layer (JavaFX):** Handles user interaction and displays data (LoginFrame, ReservationFrame).
- ❖ **Controller Layer:** Bridges the UI and the Facade (e.g., LoginController, TerrainController).
- ❖ **Facade Layer:** Provides a simplified interface for the UI to access business logic (e.g., SessionFacade, RegistrationFacade).
- ❖ **Service Layer (Business Logic):** Contains the core business rules (e.g., UserManager, ReservationManager).

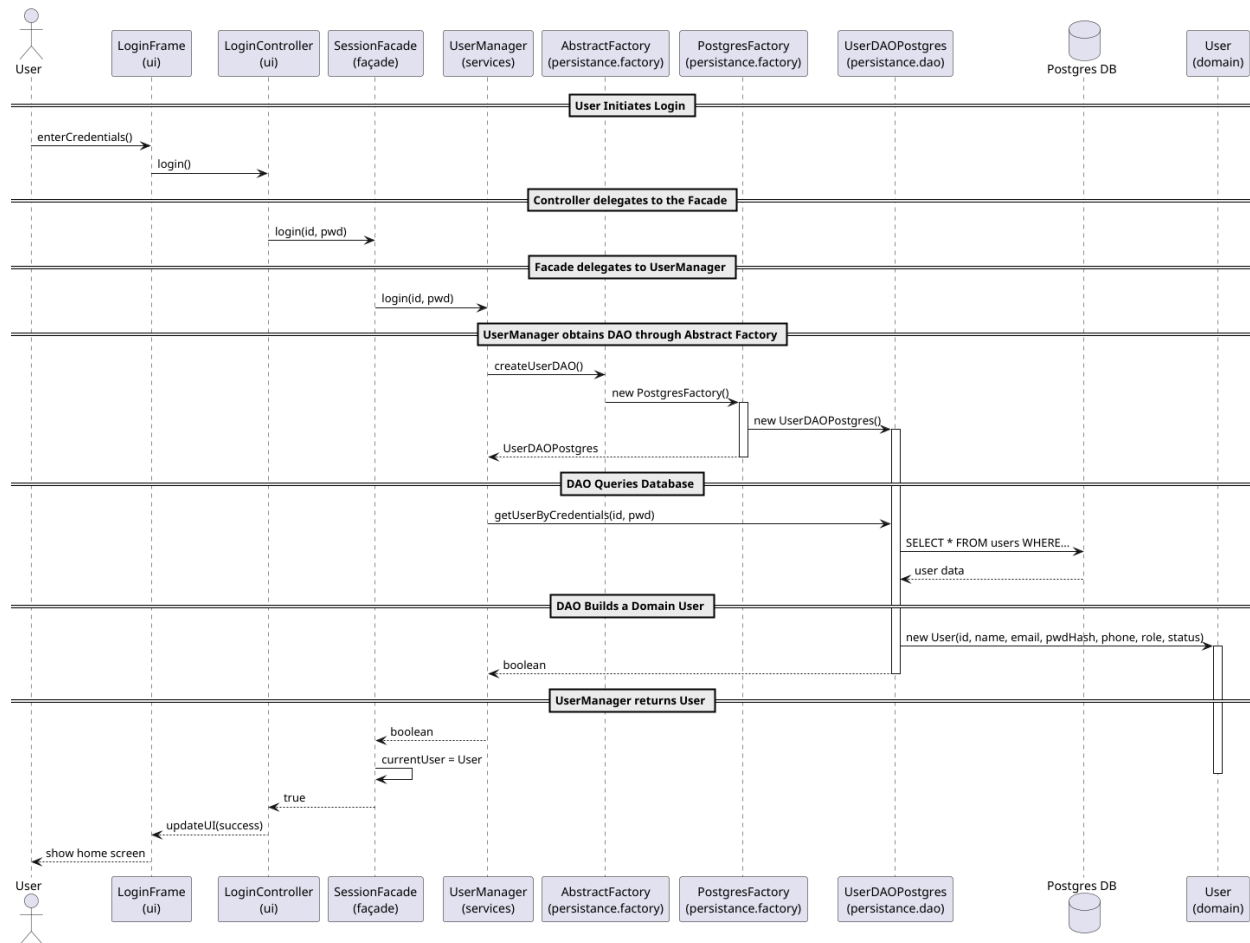
- ❖ **DAO Layer (Persistence):** Handles database interactions using the Abstract Factory pattern to support potential database switches (e.g., UserDao, PostgresFactory).

B. Prototype Diagrams:

1. Class Diagram:



2. Sequence Diagram



C. Prototype Management Report:

Date: Friday, December 12, 2025

Duration: 4 hours

Team Members: 4 people

Deadline: Undefined

Manager: BENALI Syrine

1. Project Overview

Our team is developing a Java application that serves as an intermediary platform connecting three main user groups: clients, tournament organizers, and sports facility owners. The platform allows clients to book sports facilities, participate in tournaments, and rent or buy sports equipment. Facility owners can manage their properties, define availability, and sell equipment. Tournament organizers can create and manage sporting events.

2. Objectives of the Session

According to the phase instructions, the team was required to:

1. Create the class diagram using a UML tool capable of separating the model from the diagram and generating Java code (StarUML in our case).
2. Create the sequence diagram using a UML tool (StarUML in our case).
3. Choose the technologies that we will be using and distribute the different responsibilities (JavaFX, GitHub, Database, ...).
4. Code the login prototype for our application.
5. Prepare a written document including the class diagram and sequence diagram.

3. Process Followed during the Session

3.1. Preparing the Environment

Each team member installed and tested the required tools (PlantUML, project documents, guidelines). This ensured everyone could work on the same model structure and export consistent diagrams/code.

3.2. Class Diagram Preparation

For the login, we used the professor's class diagram as a base and built on it.

For each class, we extracted:

- Potential attributes (that would be compatible with JavaFx for the UI part)
- Required associations
- Necessary packages for the class separation (UI, BL, DAO)

3.3. Reviewing Course Material

We referred to class extraction guidelines from the lectures to avoid duplicating responsibilities or creating procedural-style classes.

This helped distinguish:

- **Classes** (LoginFrame, User, SessionFaçade, UserManager, ...)
- **Value types** (DateTime, price fields, IDs...)
- **Enumerations** (Statuses, Payment Methods...)
- **Associations**

3.4. Building the UML Class Diagram

We then moved to PlantUML to construct the final domain model:

- All classes were added with attributes and types.
- Bi-directional relationships were resolved or simplified when unnecessary.
- Packages were carefully chosen to well divide the classes

This step required several iterations to harmonize naming conventions and remove redundancies.

3.5. Building the UML Sequence Diagram

We used StarUML to create the sequence diagram to create the sequence diagrams, focusing primarily on the Login process to guide our prototype implementation.

- **Layer Interaction:** The diagrams visualized the interaction between the UI Layer (LoginFrame), the Business Logic Layer (SessionFacade), and the Data Access Layer (UserManager/UserDAO).
- **Message Flow:** We defined the precise order of messages, ensuring that the UI never communicates directly with the database, strictly adhering to the separation of concerns.

3.6. Technology Selection

Before starting the implementation, we agreed on the technology stack and divided responsibilities:

- **Language & UI:** Java with JavaFX was selected for its robust GUI components and FXML support.
- **Version Control:** GitHub was chosen for source code management, with a clear branching strategy (main, dev, feature-branches).
- **Database:** We opted for a relational database (MySQL) connected via JDBC.
- **Architecture:** We enforced the DAO (Data Access Object) pattern to decouple business logic from data persistence.

4. Work Distribution

- **Wassim:** Database
- **Fatima:** Github, Façade implementation
- **Ismael:** Services and domain implementation
- **Syrine:** JavaFx and UI implementation

Everyone participated in reviewing both diagrams before exporting them.

5. Tools and Deliverables

5.1. UML Modeling Tool

- **StarUML:** For creating Class and Sequence diagrams.
- **JavaFX:** For the Graphical User Interface.
- **GitHub:** For version control and collaboration.
- **IntelliJ IDEA:** IDE used for Java development.
- **PostgreSQL:** For the database backend.

5.2. Deliverables Produced

- Login UML class diagram (final version integrated in the report)
- Login UML sequence diagram (final version integrated in the report)
- Java code for the login prototype
- Written report describing the model and process

6. Next Steps and Pending Items

After completing this phase, the next objective will be to apply the same process for the rest of the use cases.

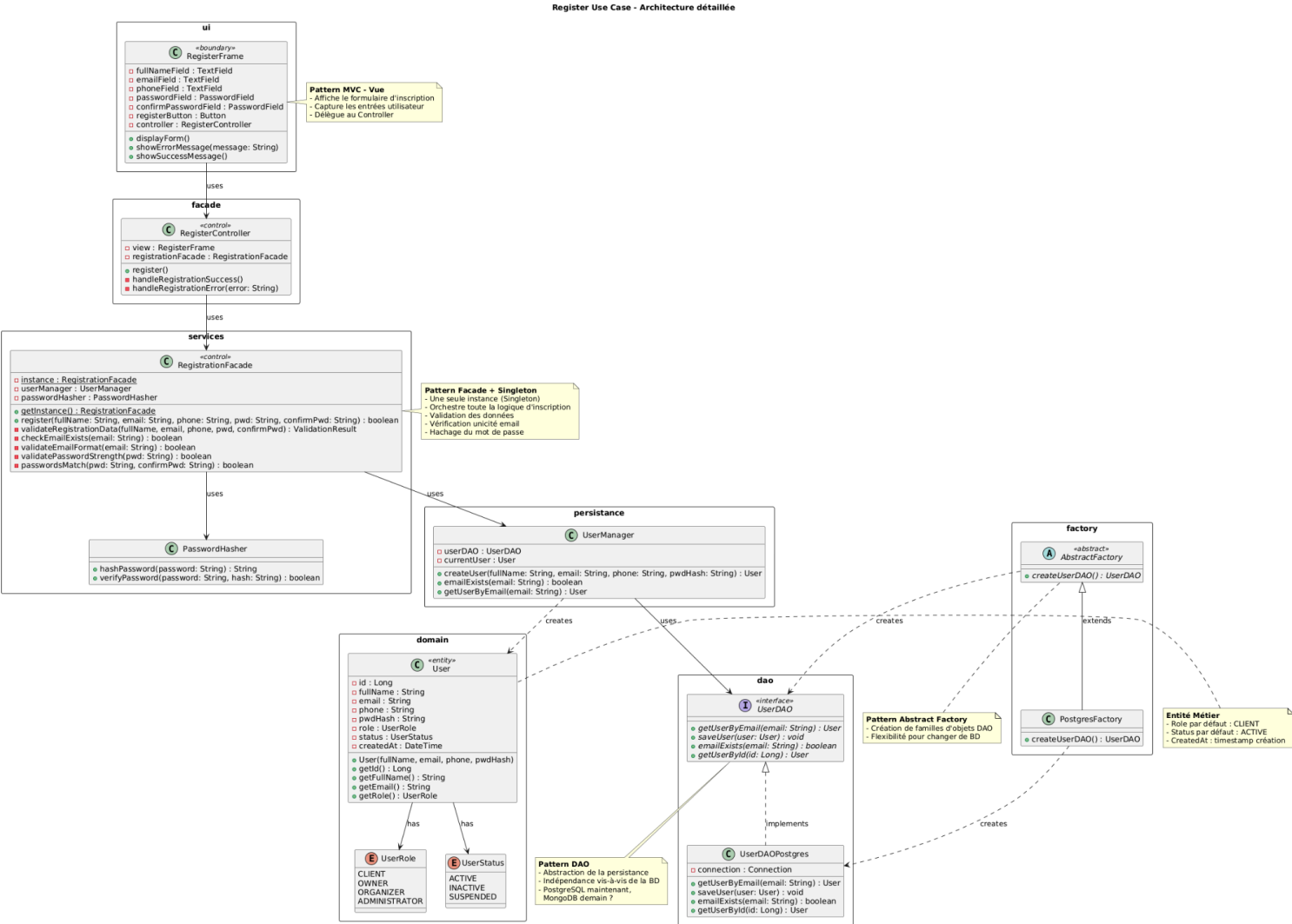
7. Conclusion

This session marked a critical transition from analysis to implementation. By successfully modeling the dynamic behavior of the Login use case and delivering a working JavaFX prototype, the team has validated the chosen architecture (DAO/Facade). The clear distribution of roles (UI, Logic, Data, Testing) proved effective and will be maintained for the upcoming development cycles.

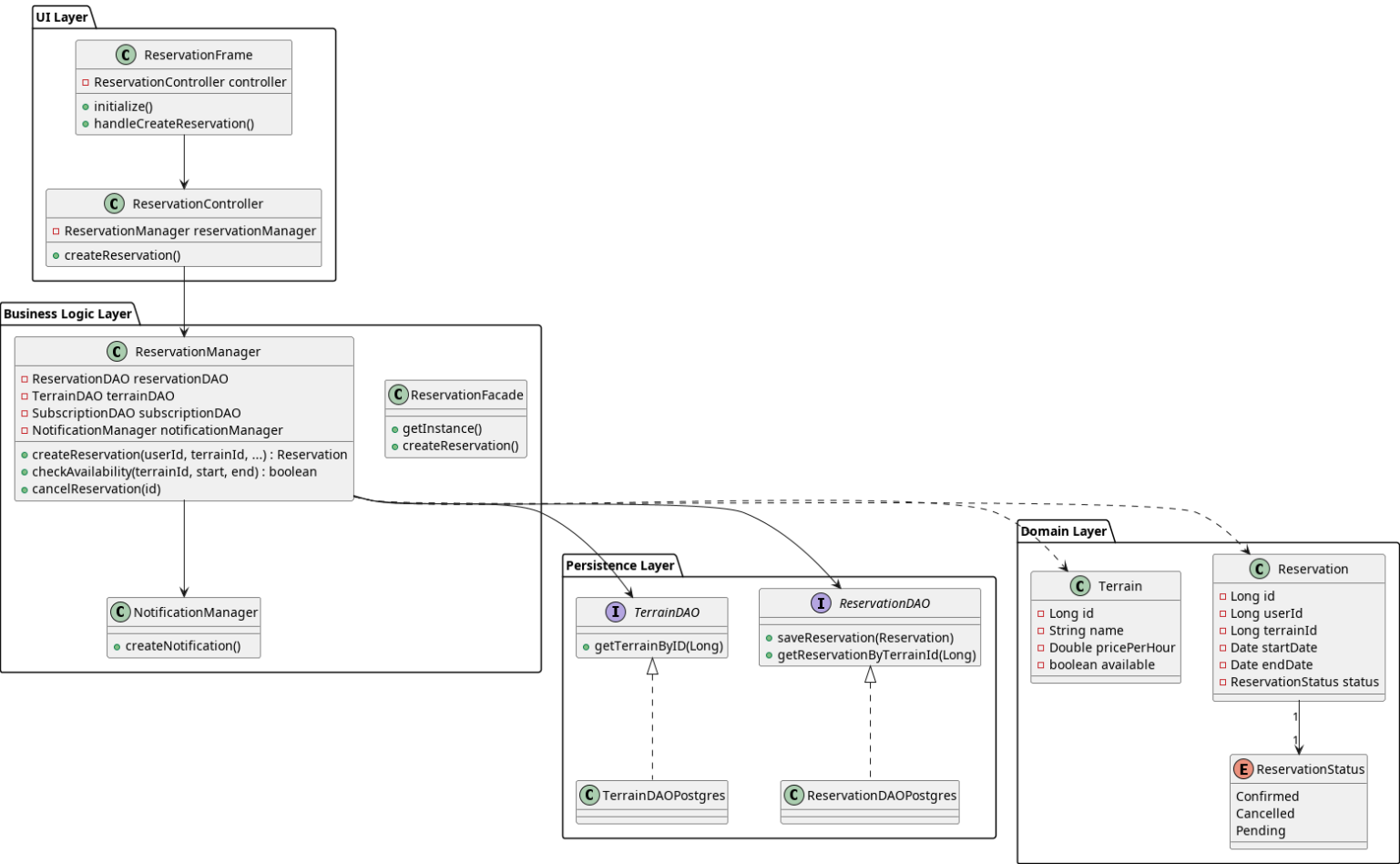
IV. First Sprint:

A. Class Diagrams:

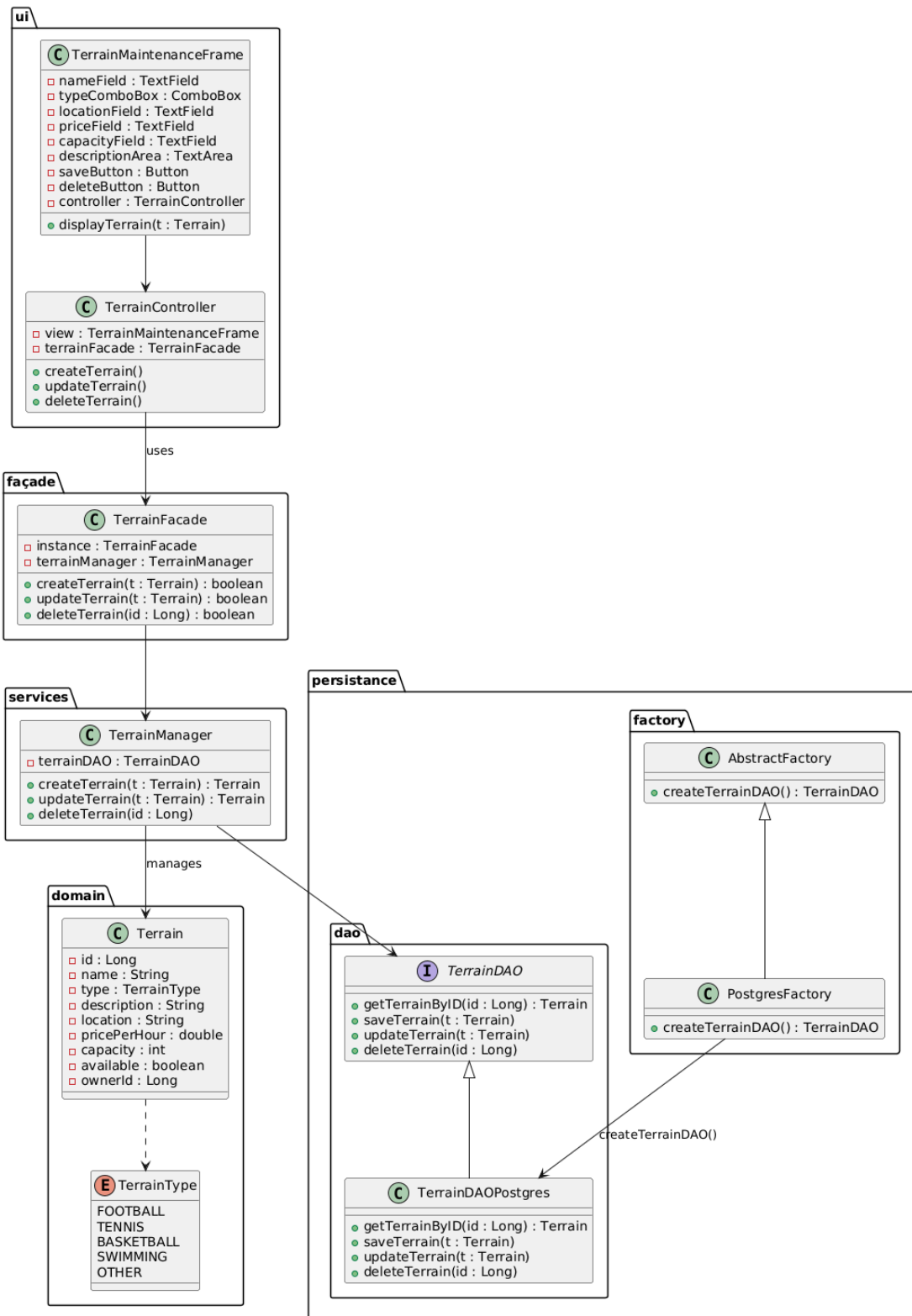
1. Authentication Management:



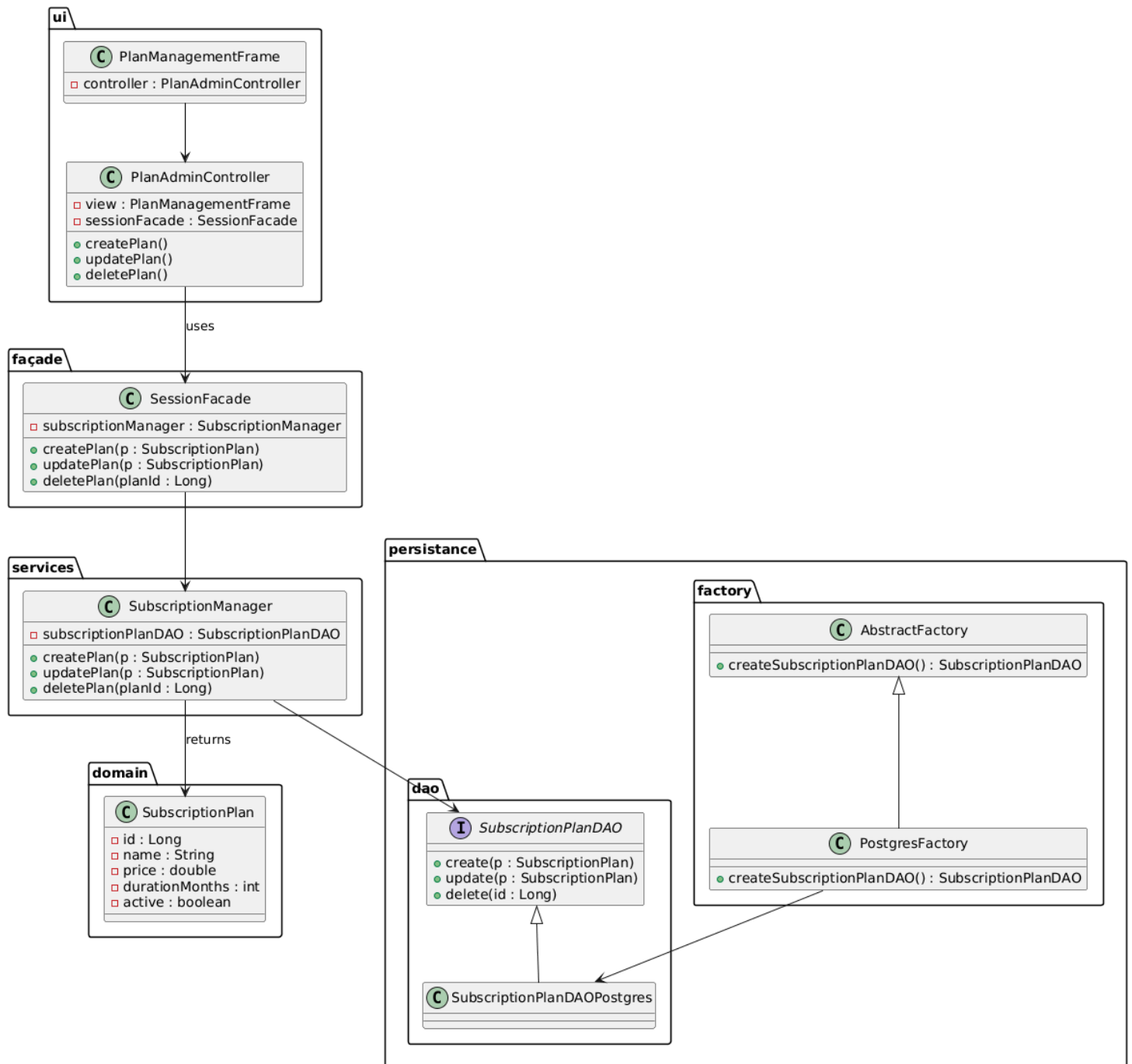
2. Reservation Management:

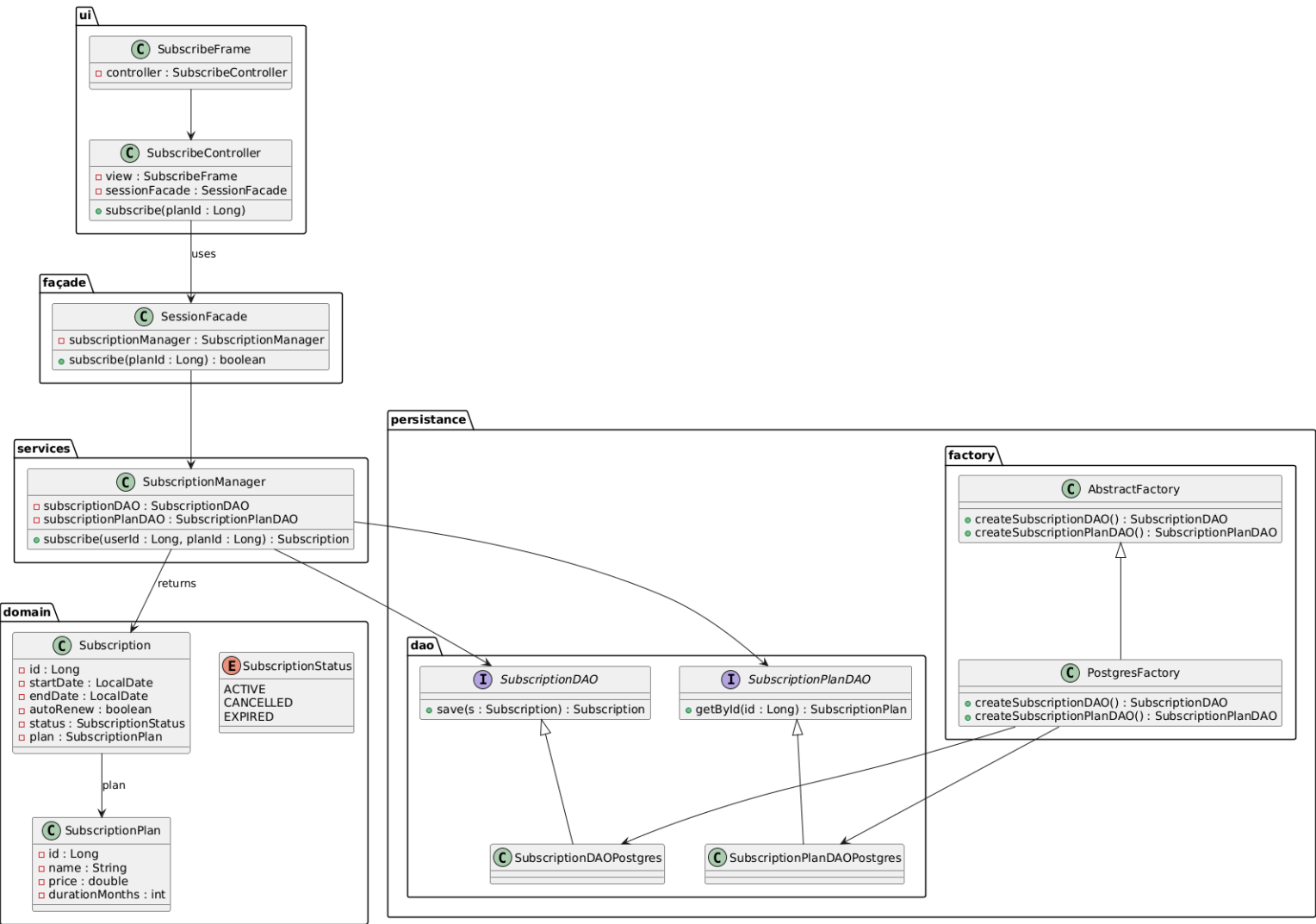


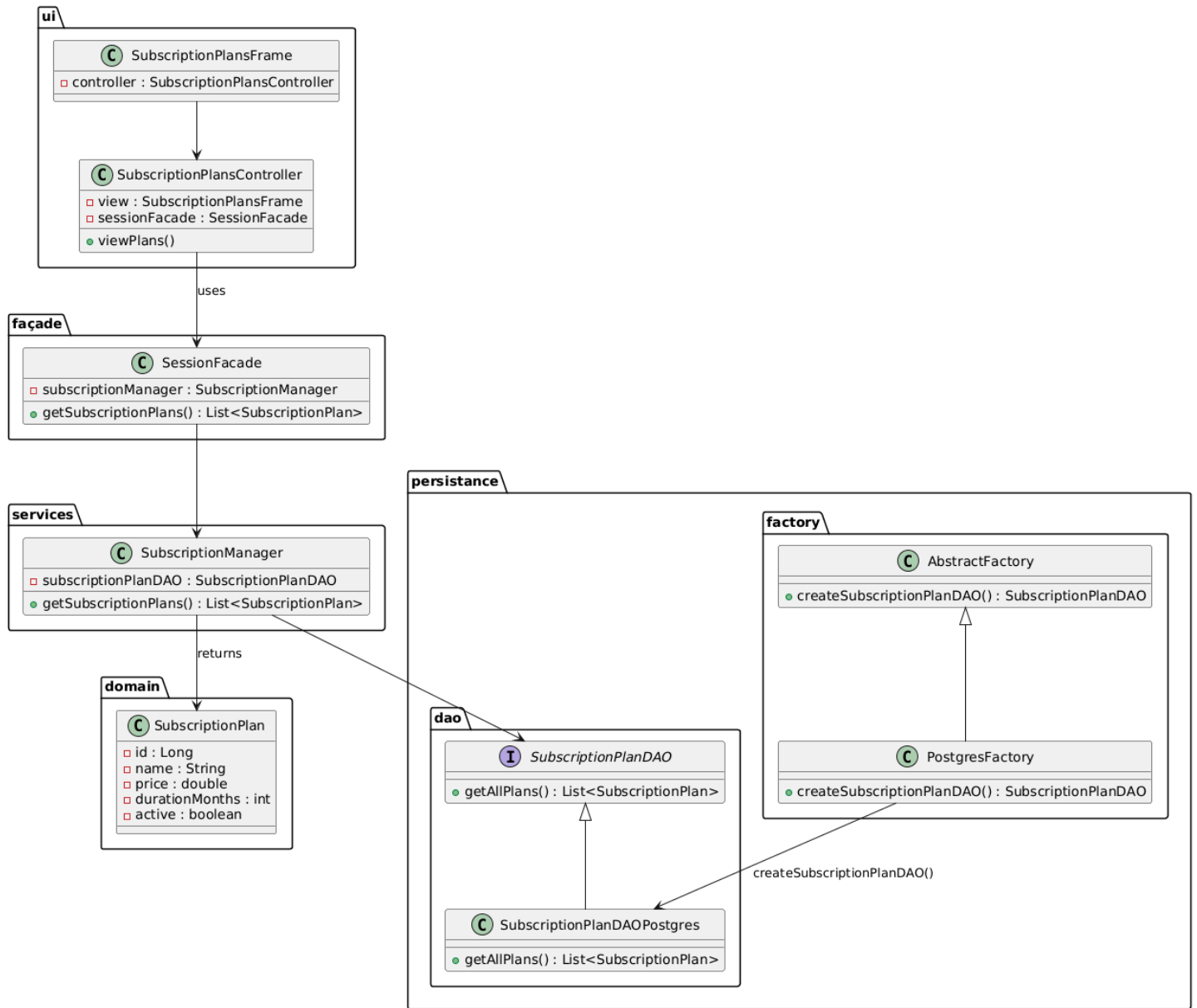
3. Terrain Management:

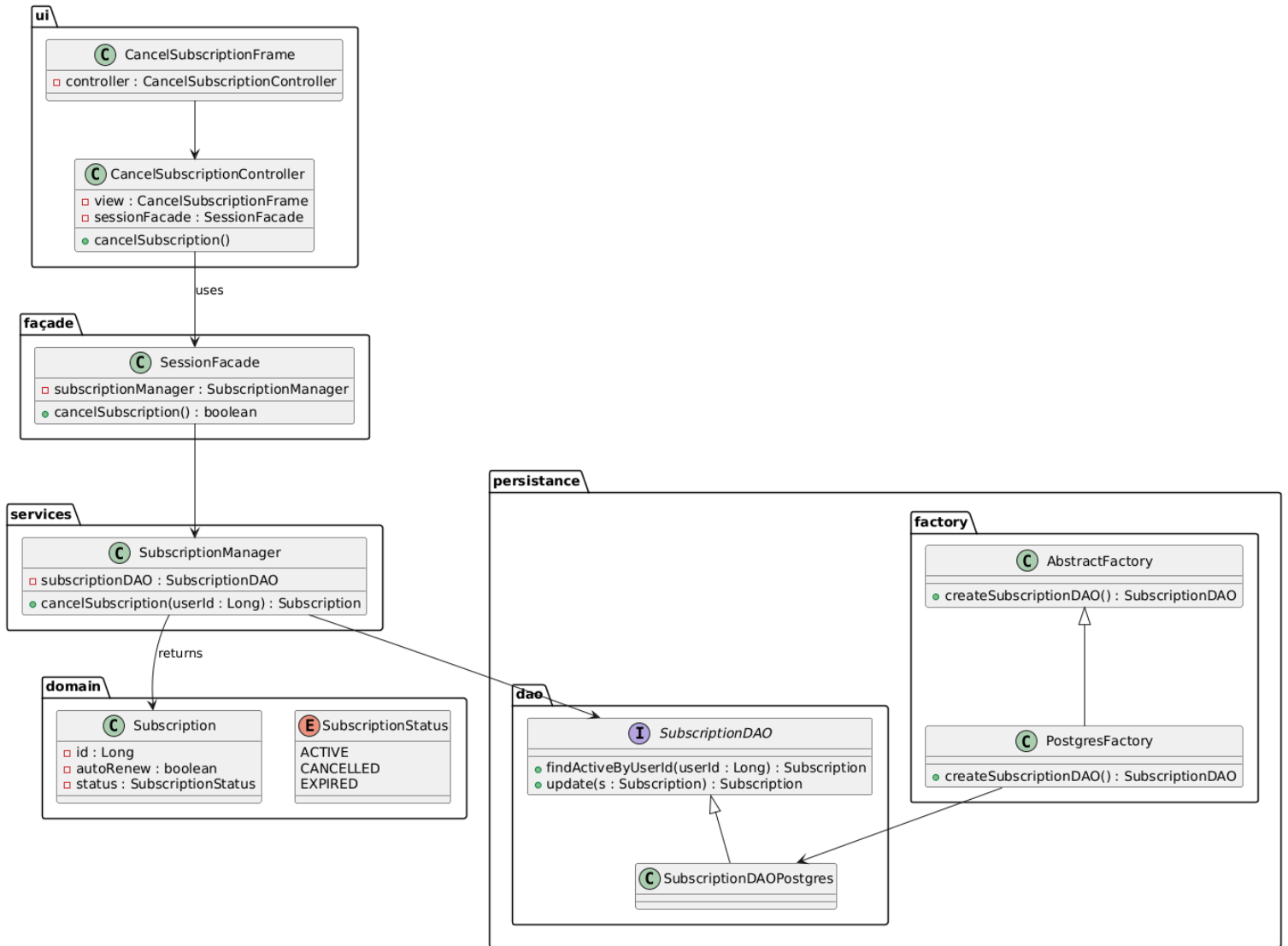


4. Subscription Management:





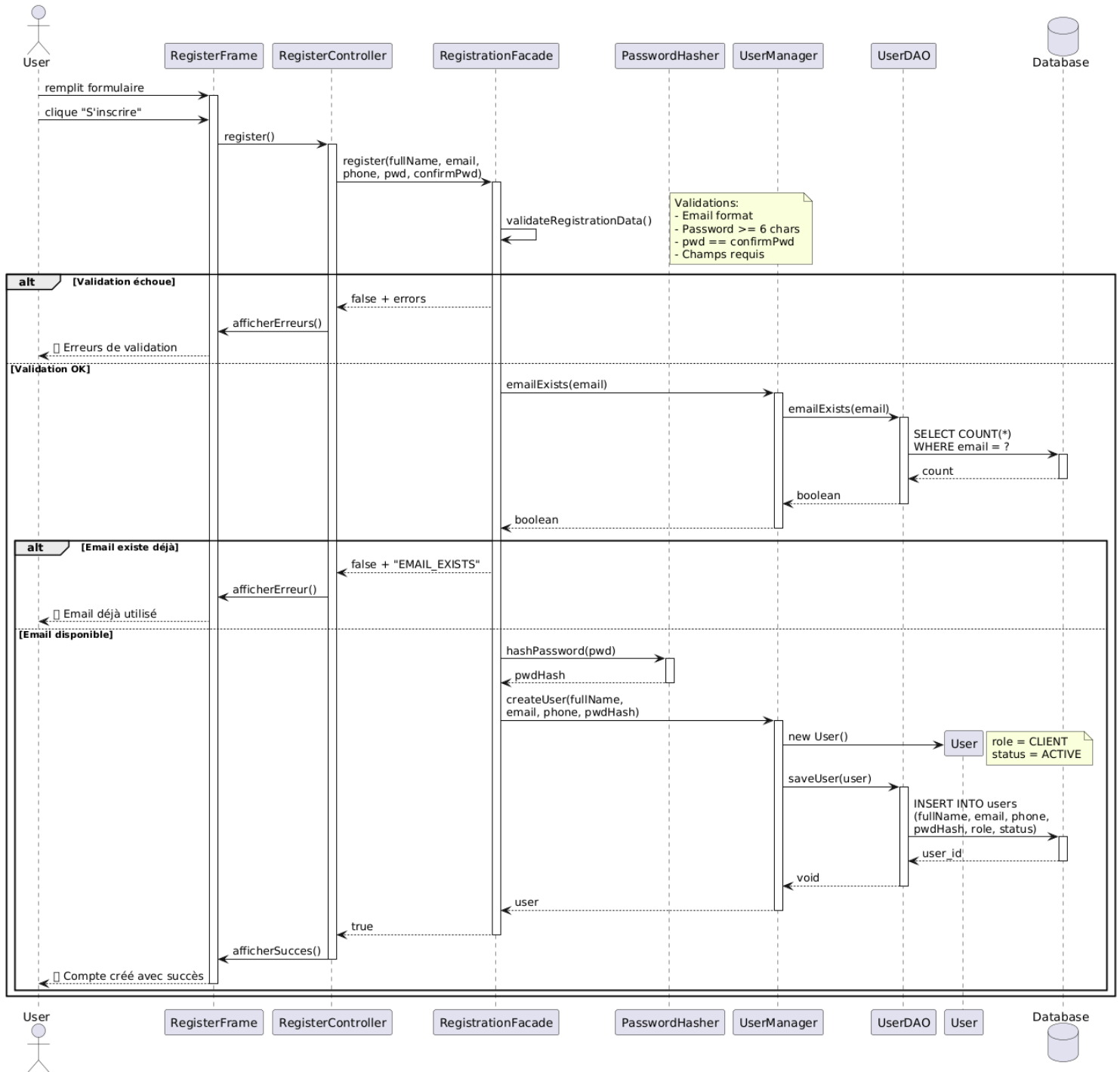




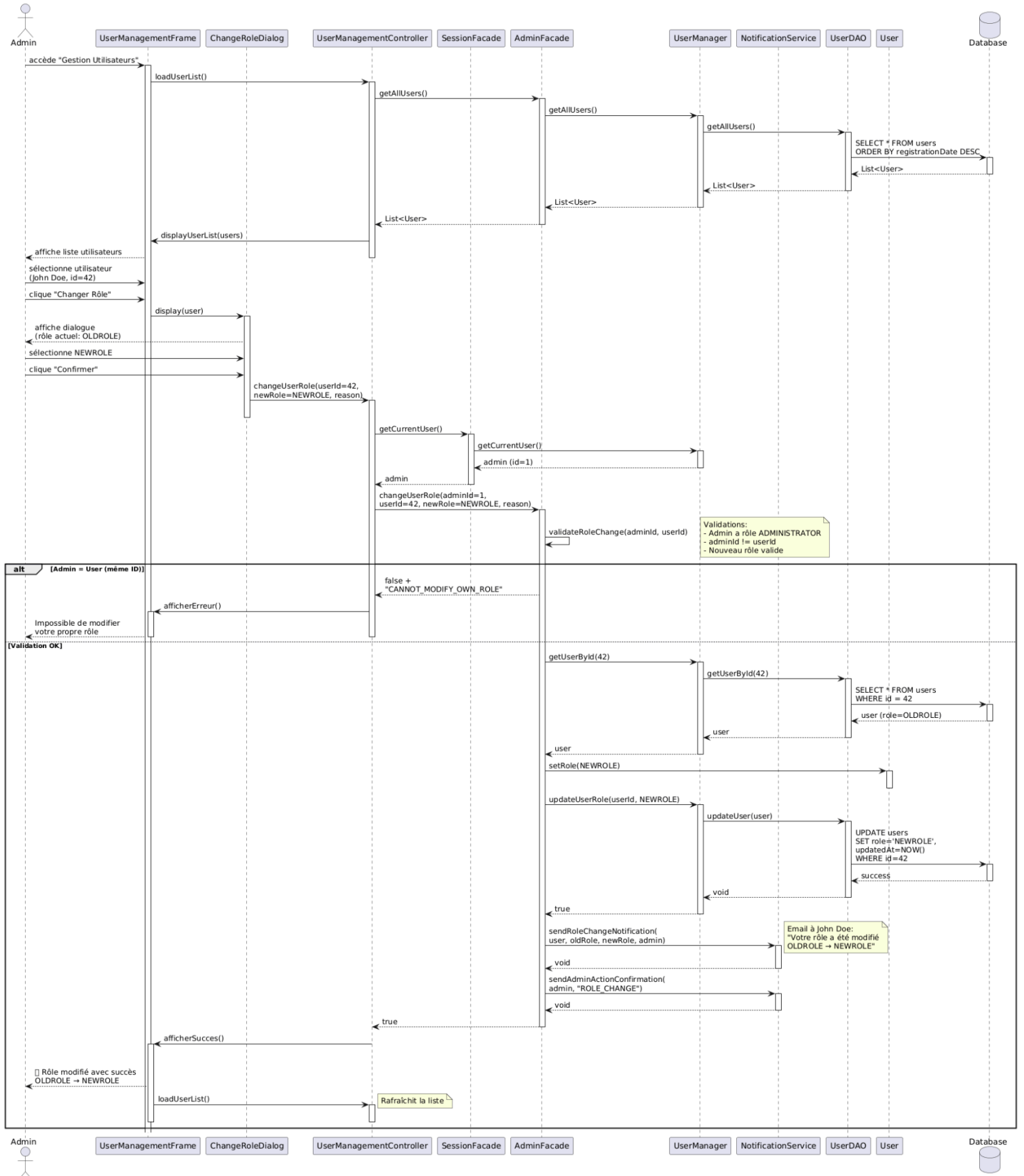
B. Sequence Diagrams:

1. Authentication Management:

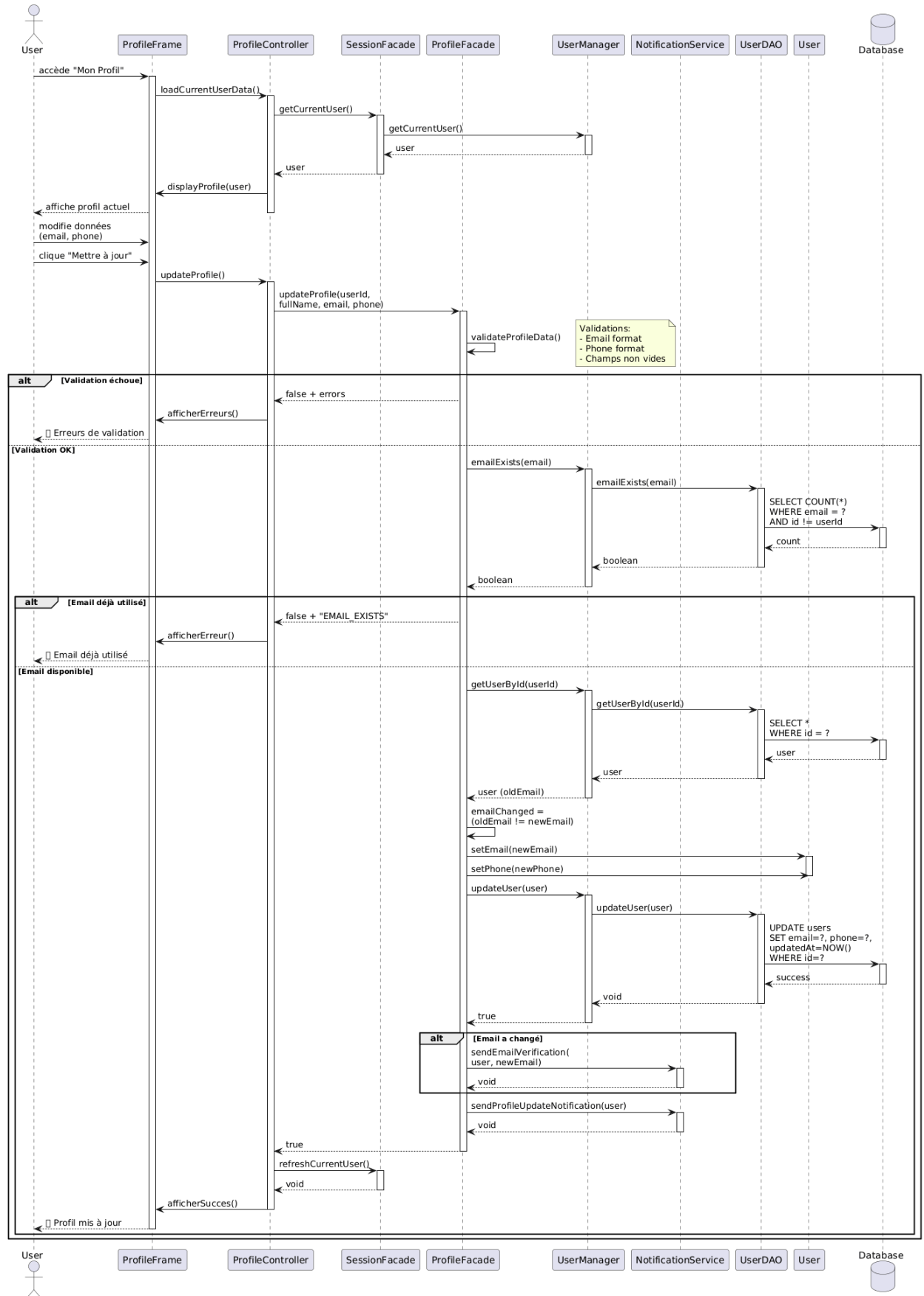
Register Use Case - Diagramme de Séquence



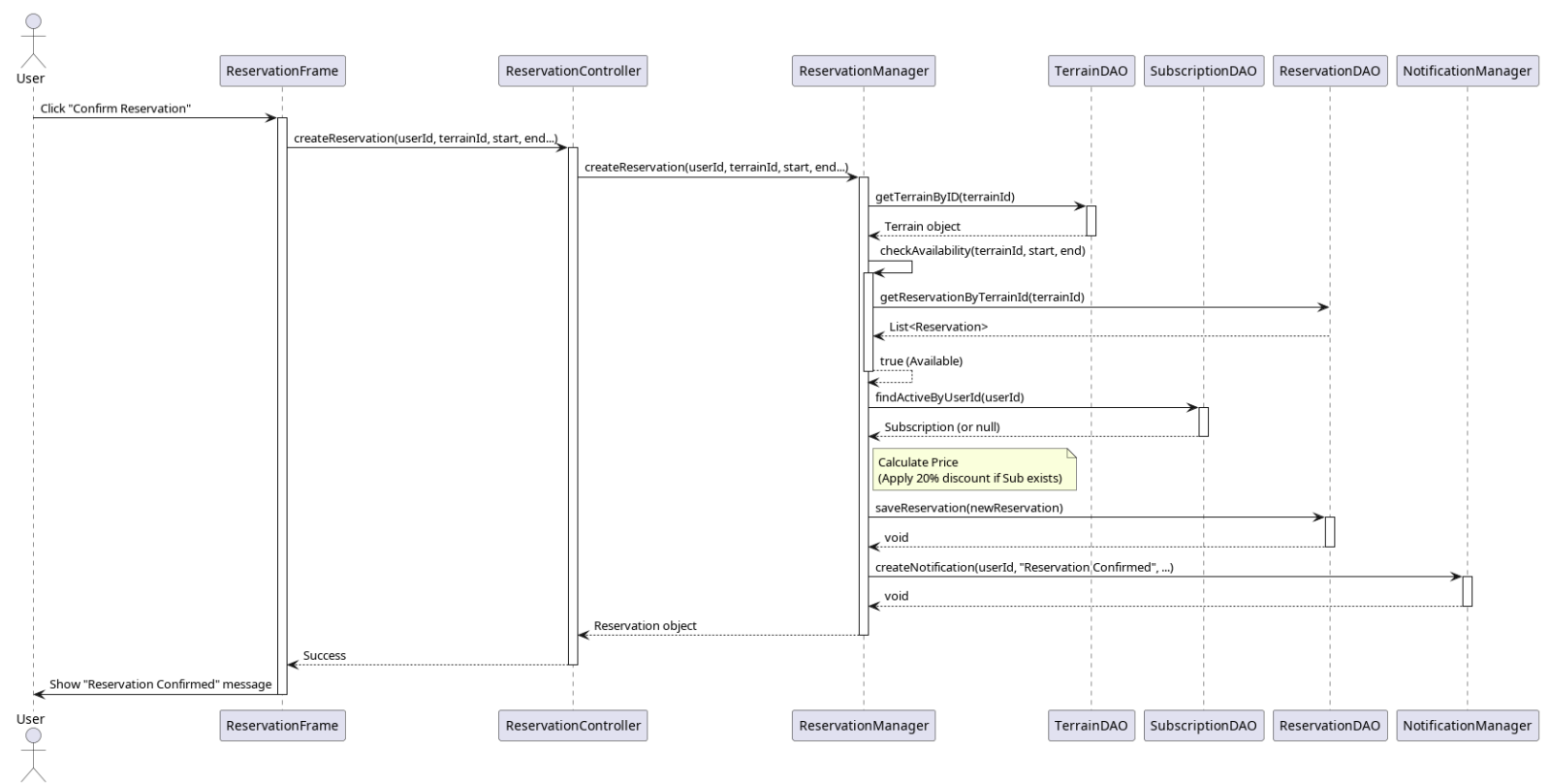
Change User Role Use Case - Diagramme de Séquence (Sans logs)



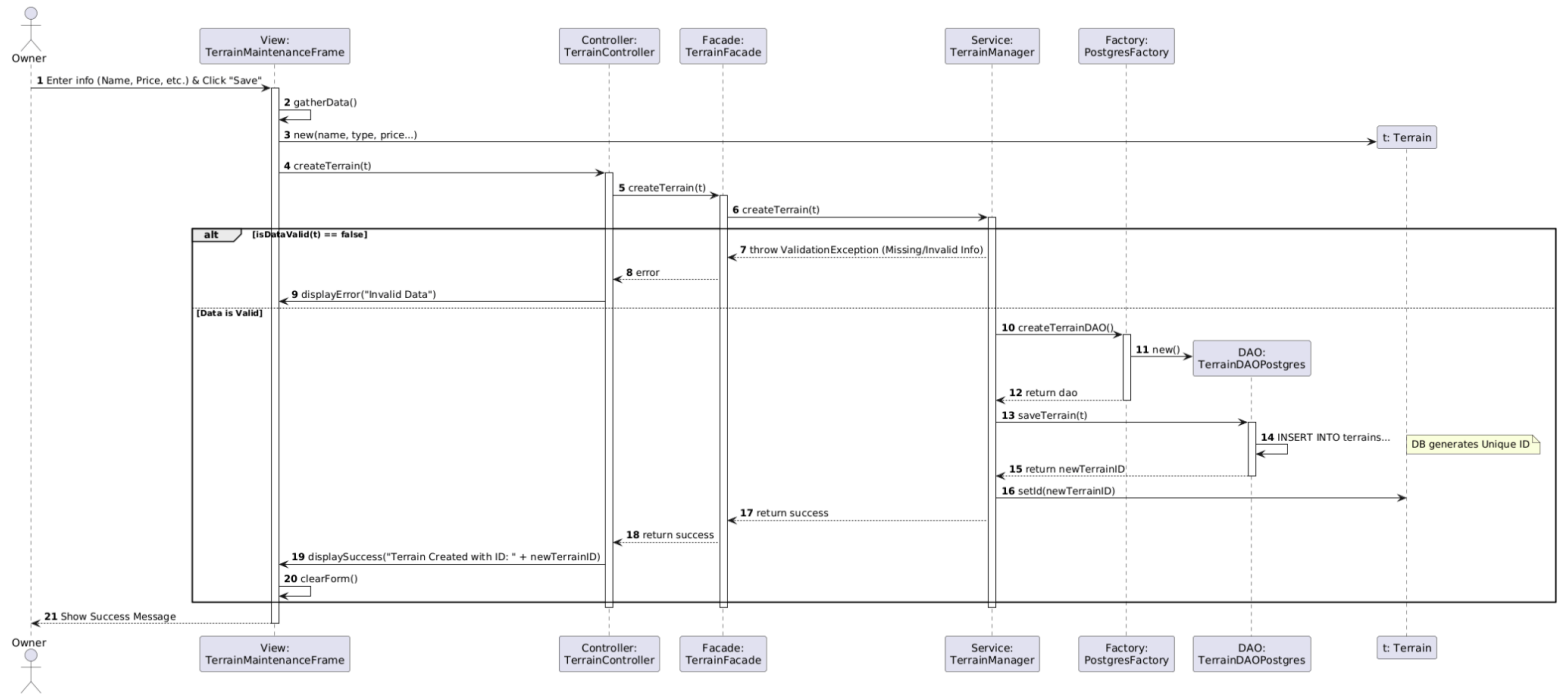
Manage Own Profile Use Case



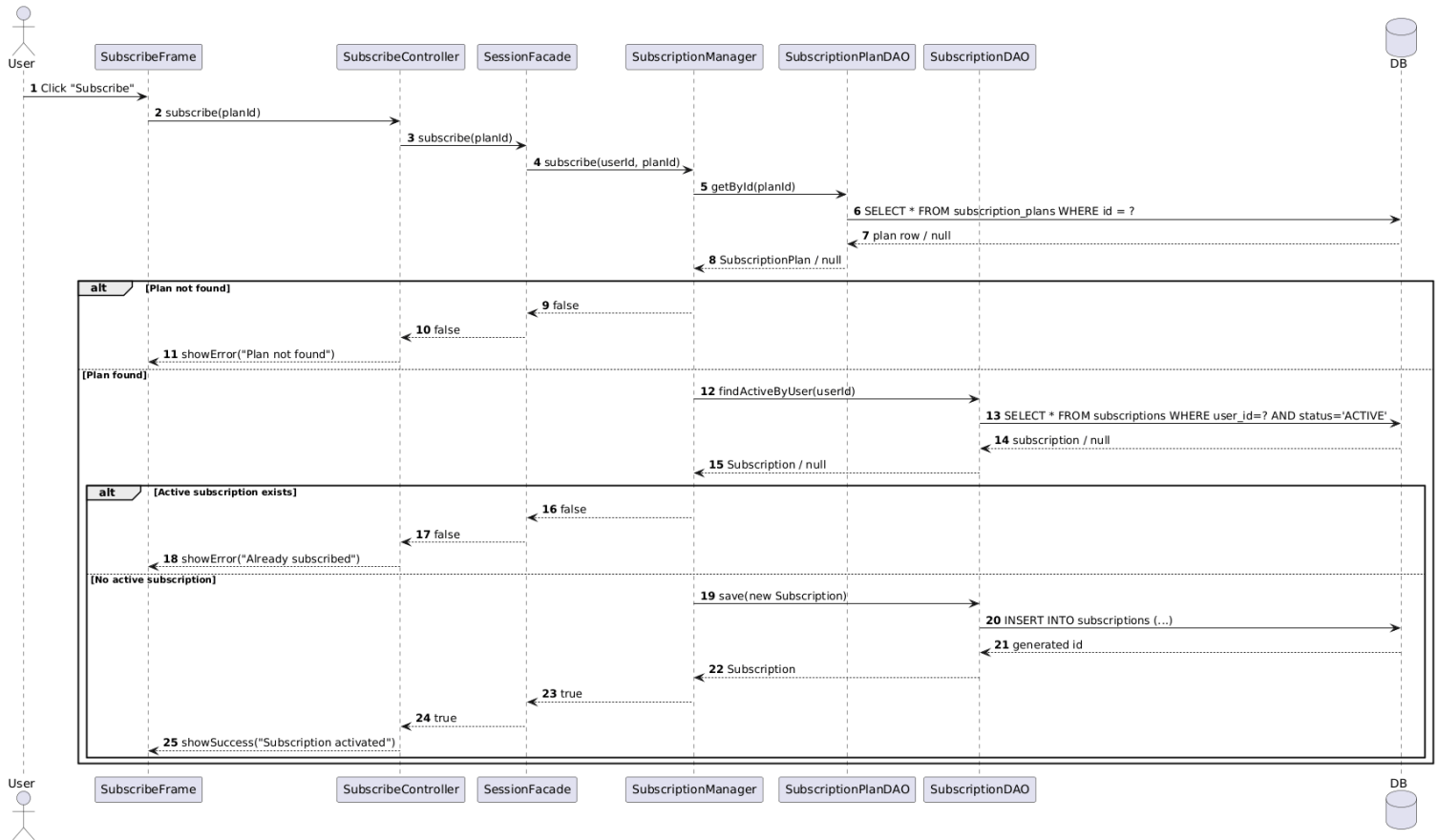
2. Reservation Management:



3. Terrain Management:



4. Subscription Management:



C. Sprint 1 Management Report:

Date: Monday, December 15, 2025

Duration: 6 hours

Team Members: 4 people

Deadline: Monday, January 5, 2026

Manager: BEN AYED Ismaël

1. Project Overview

Our team is developing a Java application that serves as an intermediary platform connecting three main user groups: clients, tournament organizers, and sports facility owners. The platform allows clients to book sports facilities, participate in tournaments, and rent or buy sports equipment. Facility owners can manage their properties, define availability, and sell equipment. Tournament organizers can create and manage sporting events.

2. Objectives of the Session

According to the phase instructions, the team was required to:

1. Design UML class diagrams for **one new use case per team member** during this sprint.
2. Ensure that all designs followed the same architectural principles validated in previous sprints.
3. Implement the designed use cases in order to validate the proposed architecture.
4. Improve collaboration by distributing two use cases per team member, while focusing on the implementation of one during this sprint.
5. Prepare the application for large-scale development by clearly defining responsibilities and interfaces.
6. Set up and use a shared remote database to support development activities.

3. Process Followed during the Session

3.1. Preparing the Environment

Before starting the design phase, the team agreed to reuse the same architectural template established during the previous sprint.

3.2. Class Diagram Preparation

Each team member was assigned two use cases.

During this sprint, the team focused on **designing and implementing one use case per member**, while keeping the second use case for a future sprint.

UML class diagrams were created for the selected use cases, and sequence diagrams were produced when relevant.

The design work focused on:

- Proper separation of concerns
- Reusability of domain entities
- Consistency in naming conventions
- Alignment with Java and JavaFX constraints

Once the designs were completed, the corresponding use cases were implemented based on the UML models.

3.3. Infrastructure Decisions

To avoid inconsistencies between local environments, the team decided to use a **remote PostgreSQL database hosted on Neon**.

This ensured that all members could work with the same database structure during development.

4. Work Distribution

Responsibilities were distributed based on functional domains:

- **Ismaël (Sprint Manager):**
 - Subscription Management (Design)
 - Terrain Management (Development)
 - Notifications (next sprint)
- **Wassim:**
 - Reservation Management (Design)
 - Authentication Management (Development)
 - Tournament Management (next sprint)
 - Database setup and deployment
- **Syrine:**
 - Terrain / Installation Management (Design)
 - Reservation Management (Development)
 - Equipment Management (next sprint)
 - Merging all developed use cases
- **Fatima:**
 - Authentication Management (Design)
 - Subscription Management (Development)
 - Cart Management (next sprint)

5. Tools and Deliverables

5.1. UML Modeling Tool

- **StarUML:** For creating Class and Sequence diagrams.
- **JavaFX:** For the Graphical User Interface.
- **GitHub:** For version control and collaboration.
- **IntelliJ IDEA:** IDE used for Java development.
- **PostgreSQL:** For the database backend (with Neon).

5.2. Deliverables Produced

- **UML class diagrams for:**
 - Authentication Management
 - Terrain Management
 - Subscription Management

- Reservation Management
- UML sequence diagrams for selected use cases
- Implementation of the selected use cases
- Merging of all developed use cases into the main development branch
- Remote database configuration
- Updated design documentation

6. Process Metrics and Quantitative Overview

The design and implementation work for this sprint was conducted over two main working sessions of approximately 3 hours each. In addition, several team members continued working asynchronously during the holiday period to finalize implementation and integration tasks.

The UML design phase required approximately 2 hours and focused on the definition of class diagrams and sequence diagrams for the selected use cases. The implementation phase required more time, as it involved coding, testing, and integrating the designed use cases into the existing application architecture.

During this sprint:

- 4 major use cases were designed (Authentication, Terrain Management, Reservation, Subscription)
- 4 use cases were partially or fully implemented

7. Problems Encountered and Lessons Learned

One of the main challenges during this sprint was **coordination** between team members. Since each member designed a different use case than the one they implemented, development sometimes had to wait for all design diagrams to be completed.

Another difficulty was **merging multiple use cases**, as several core files were modified by different members, leading to merge conflicts. These issues were resolved through improved communication and a more structured merging process, highlighting the importance of early synchronization and clear responsibility management.

8. Next Steps and Pending Items

The next sprint will focus on **designing and implementing the remaining use cases that were already assigned during this sprint**.

The team will continue development based on the established UML patterns and ensure consistency between the design and the implementation.

9. Conclusion

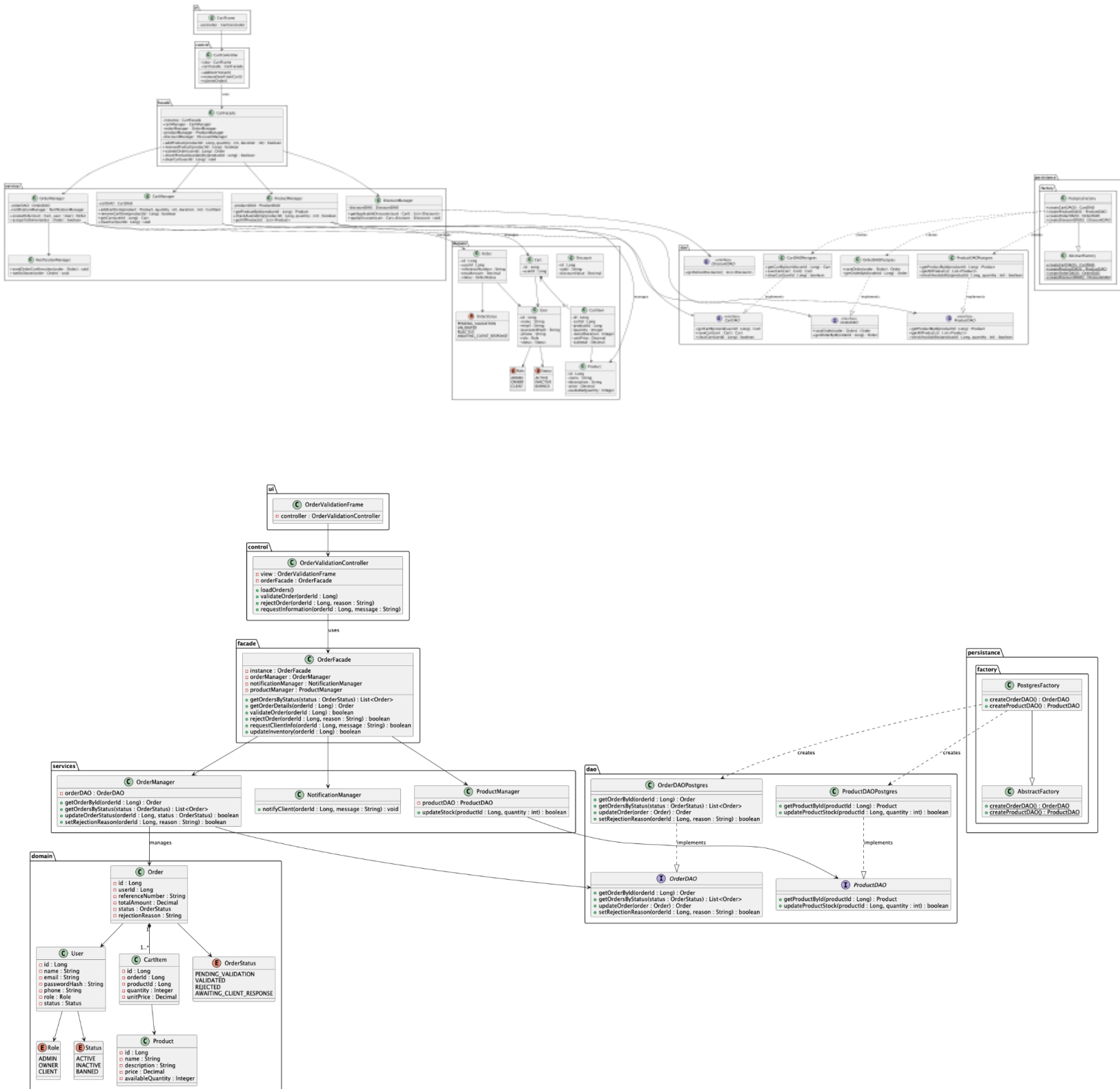
This sprint successfully extended the architectural design of the My Arena application to several key functional areas and validated the proposed architecture through implementation. By strictly reusing the design patterns established in earlier sprints, the team ensured coherence and scalability of the system.

The clear distribution of responsibilities, the integration of the developed use cases, and the use of a shared remote database environment significantly improved collaboration and prepared the team for the next development phase.

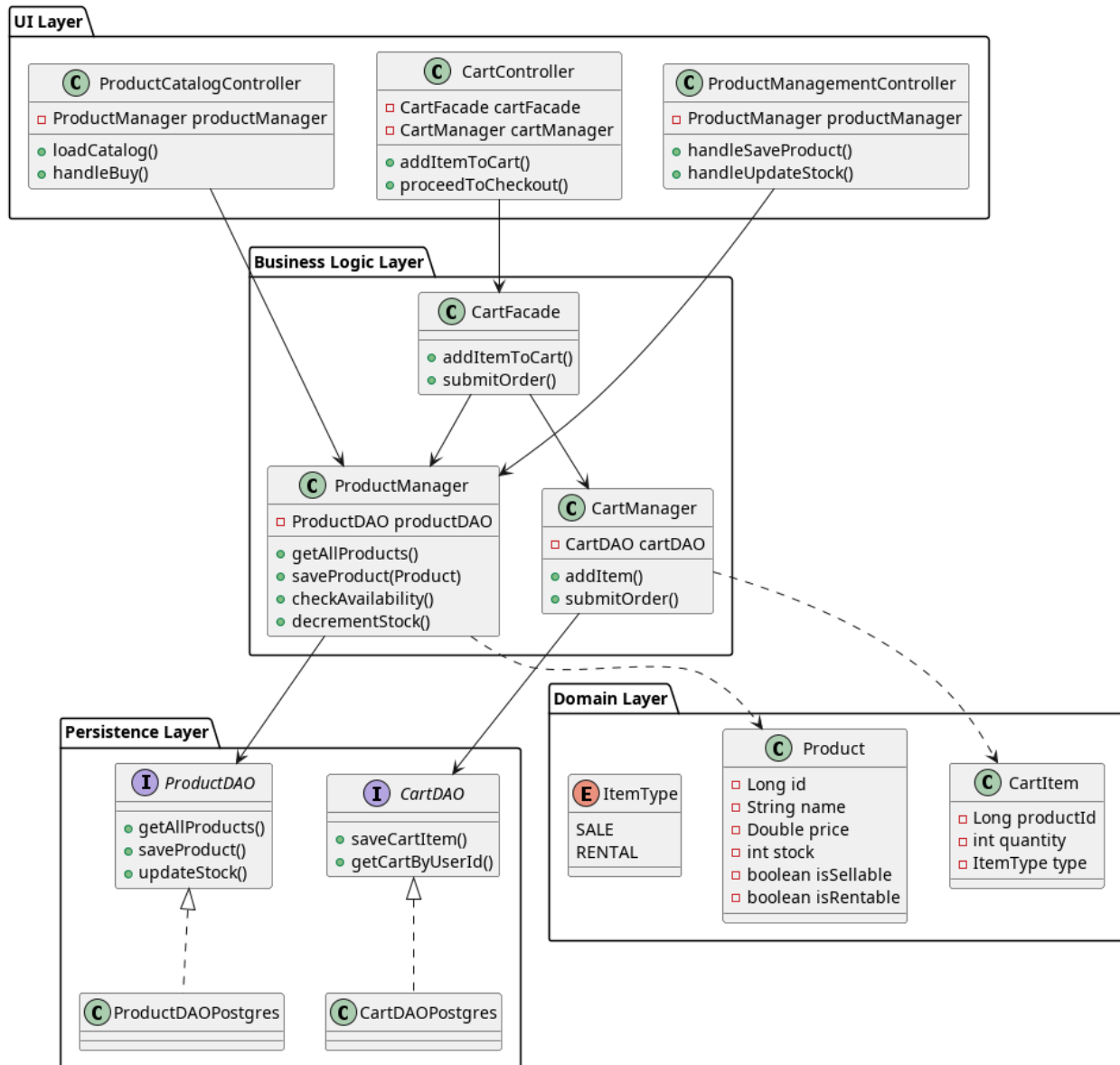
V. Second Sprint:

A. Class Diagrams:

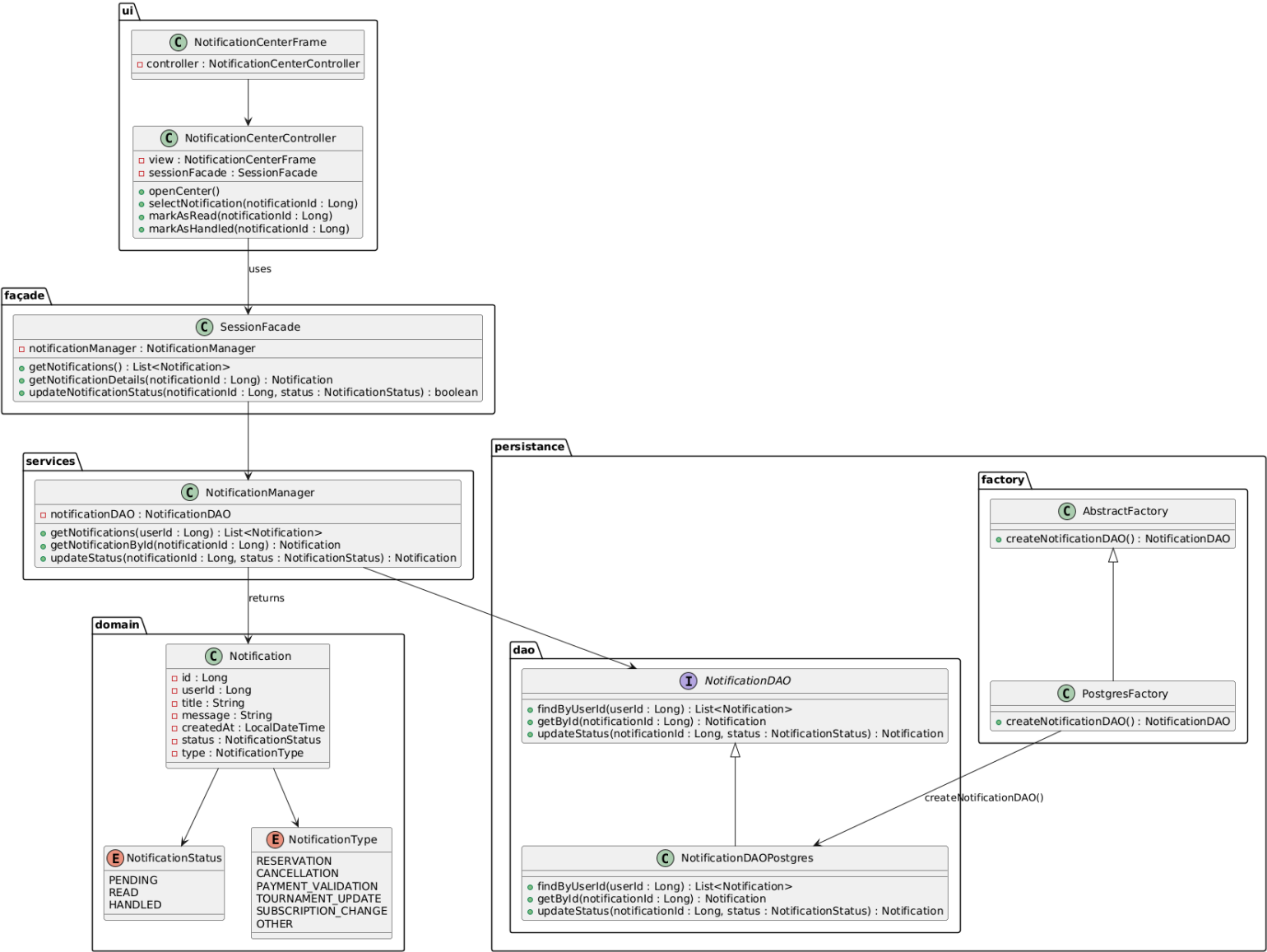
1. Cart and Order Management:

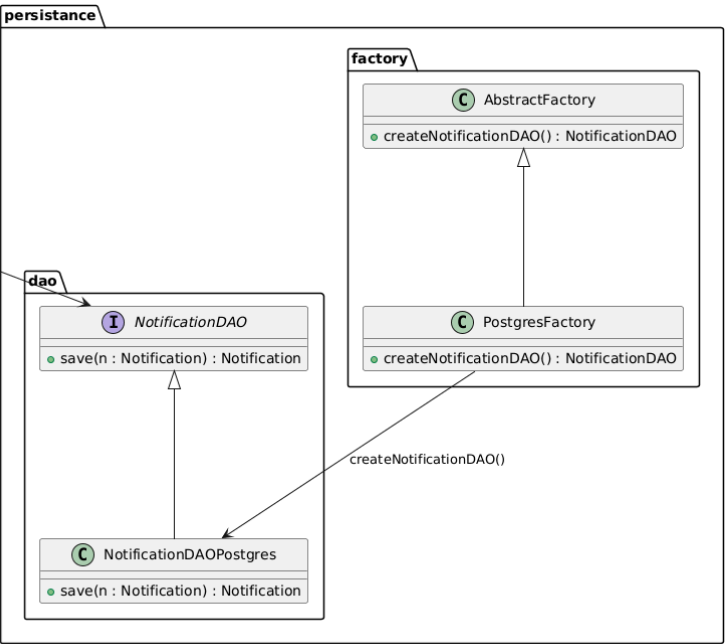
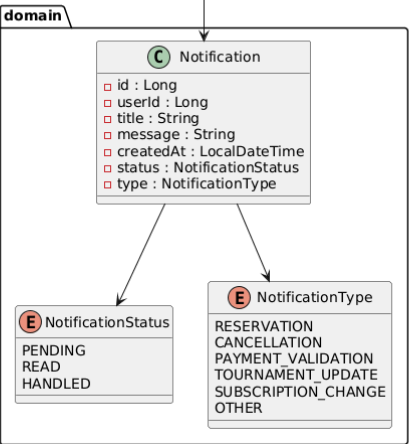
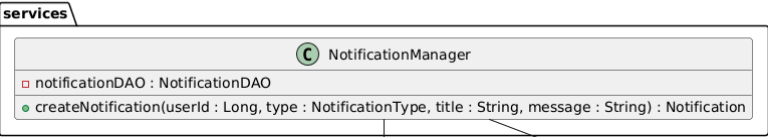


2. Equipment Management:



3. Notification Management:

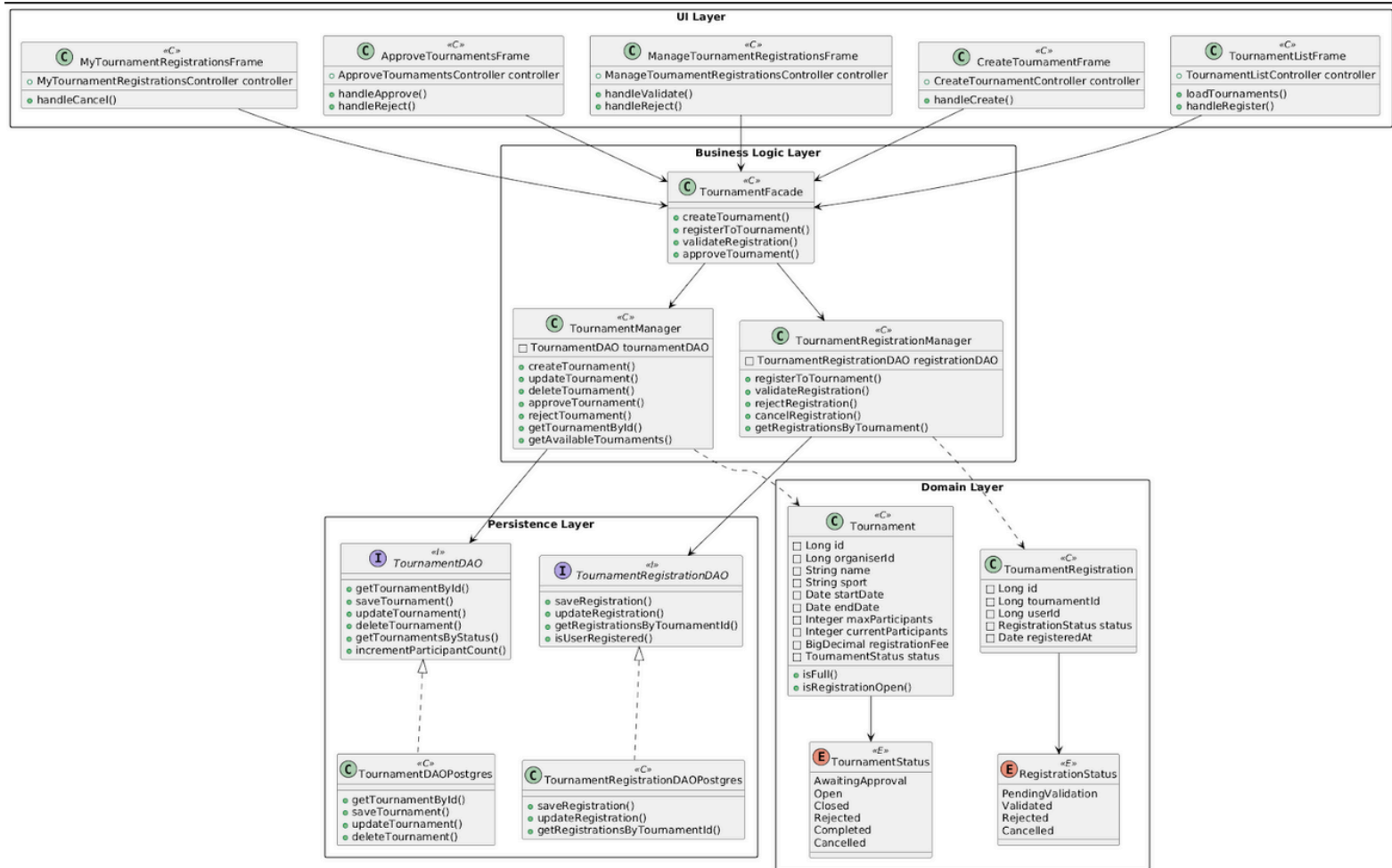




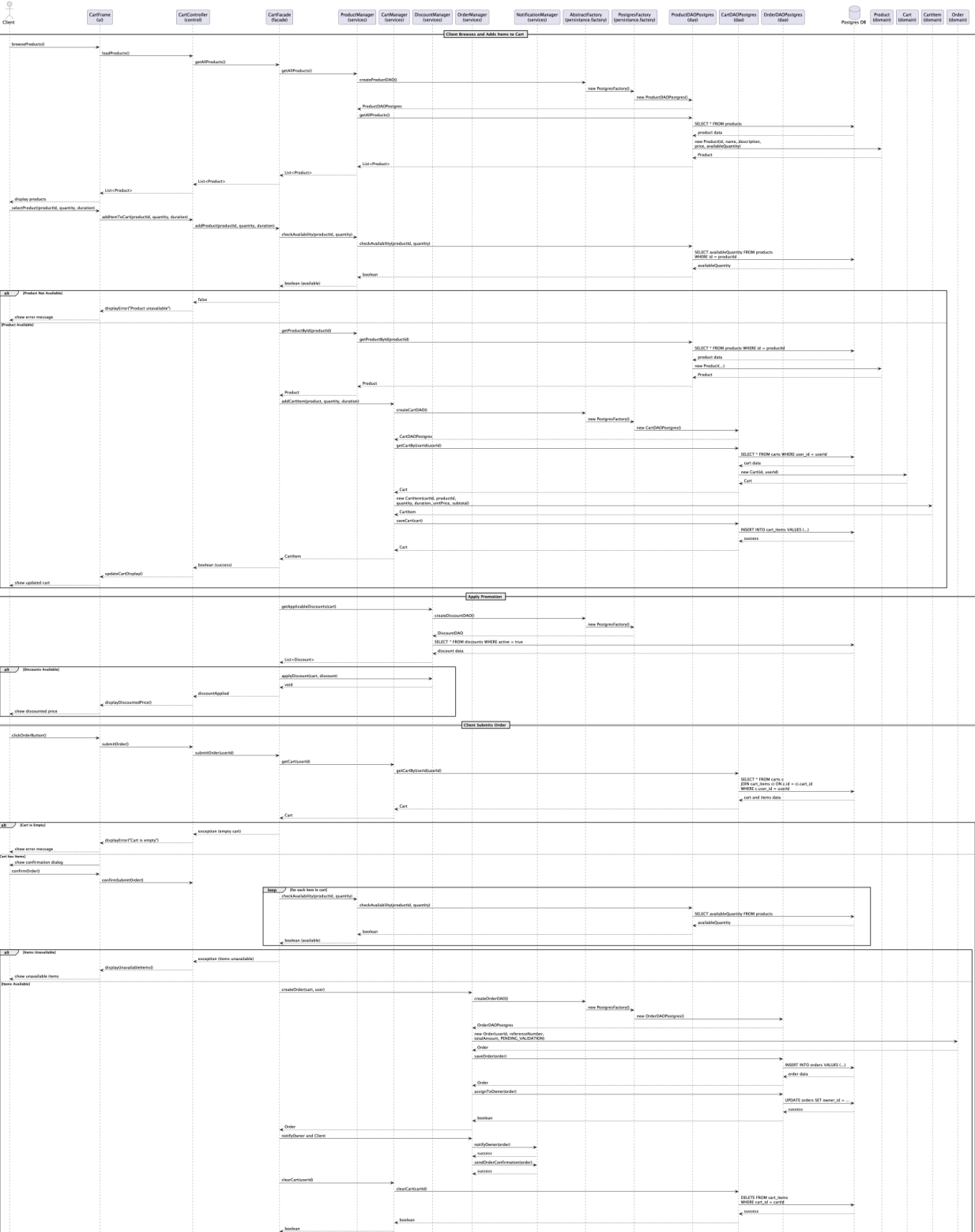
returns

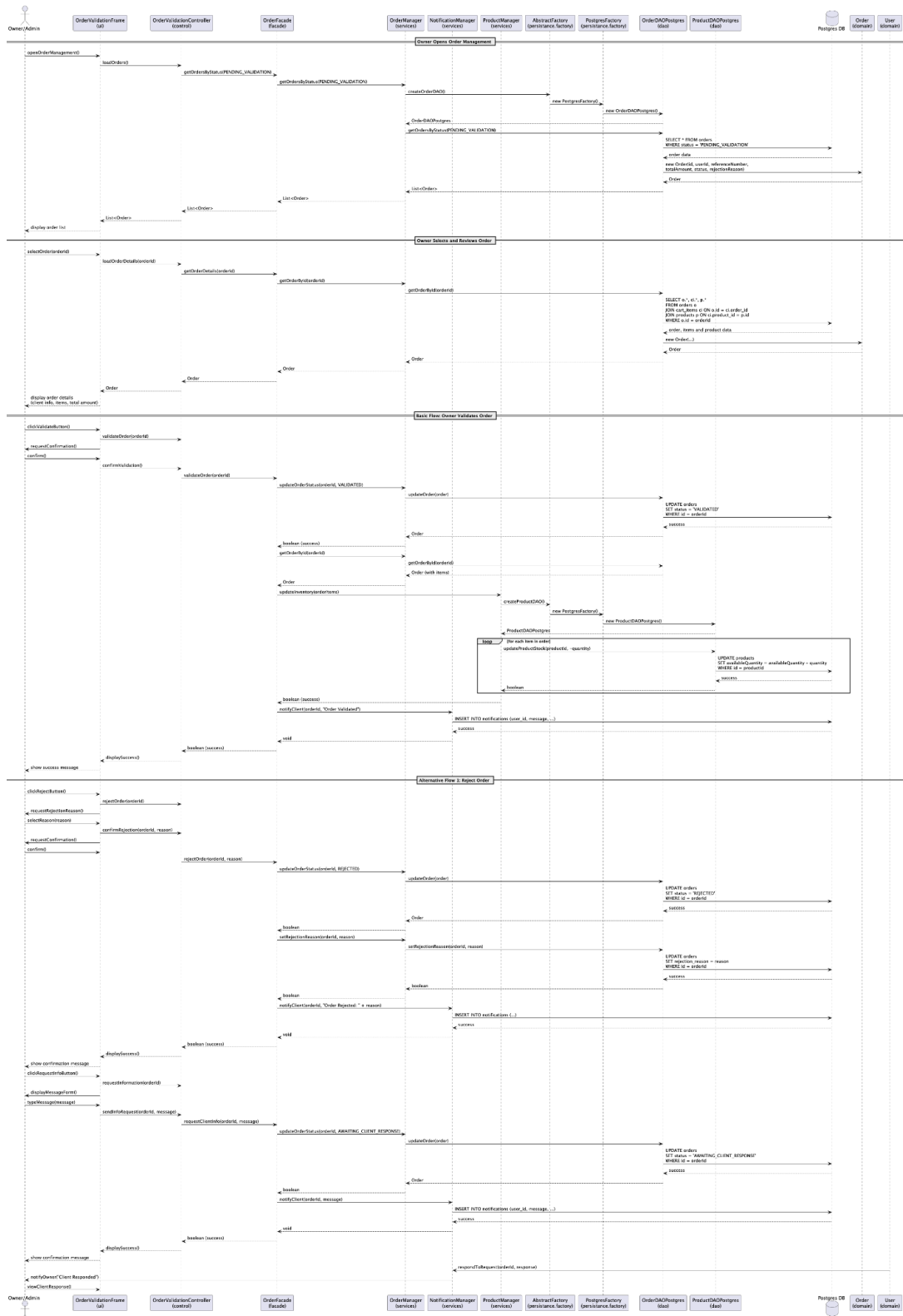
createNotificationDAO()

4. Tournament Management:

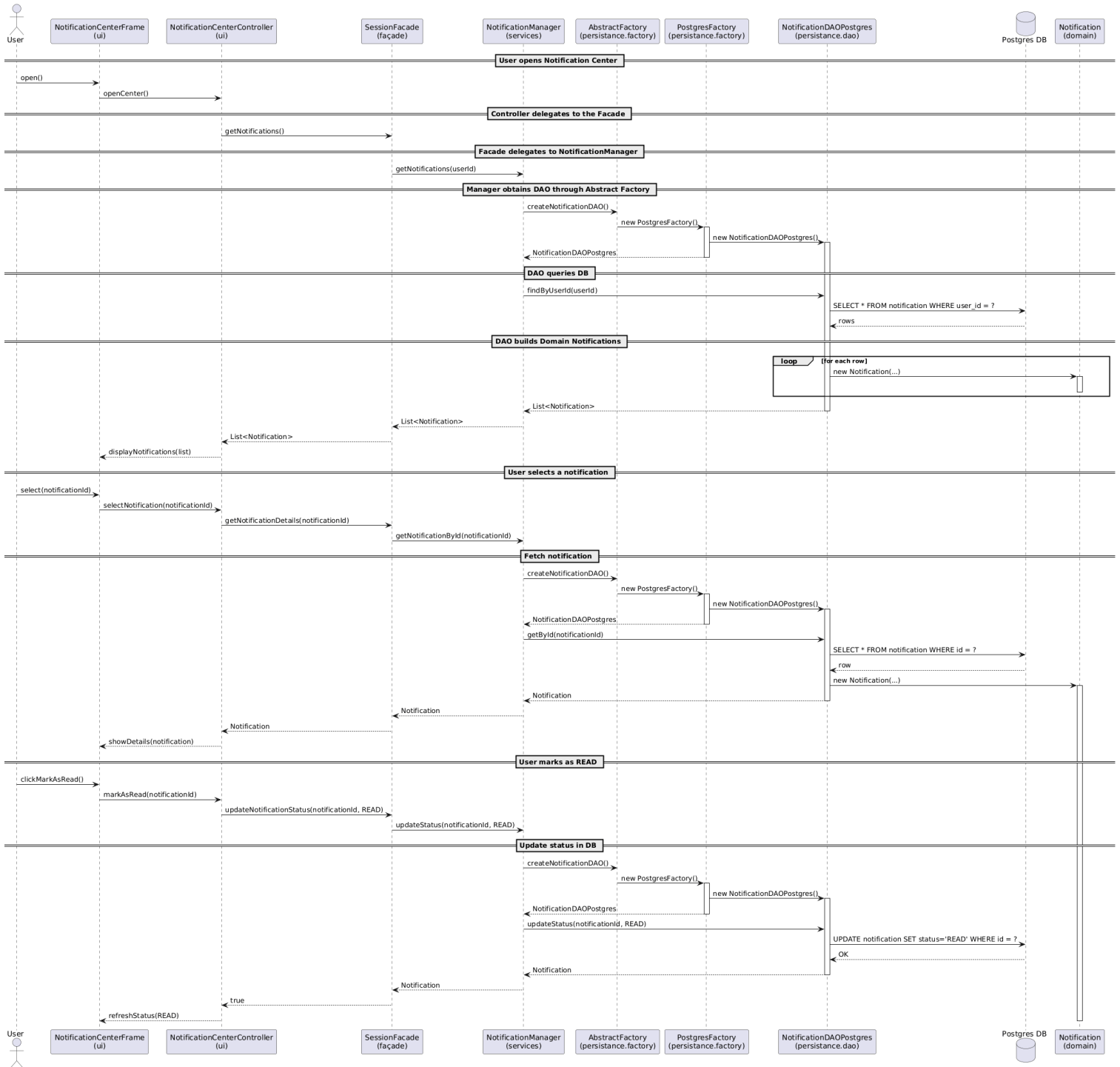


1. Cart and Order Management

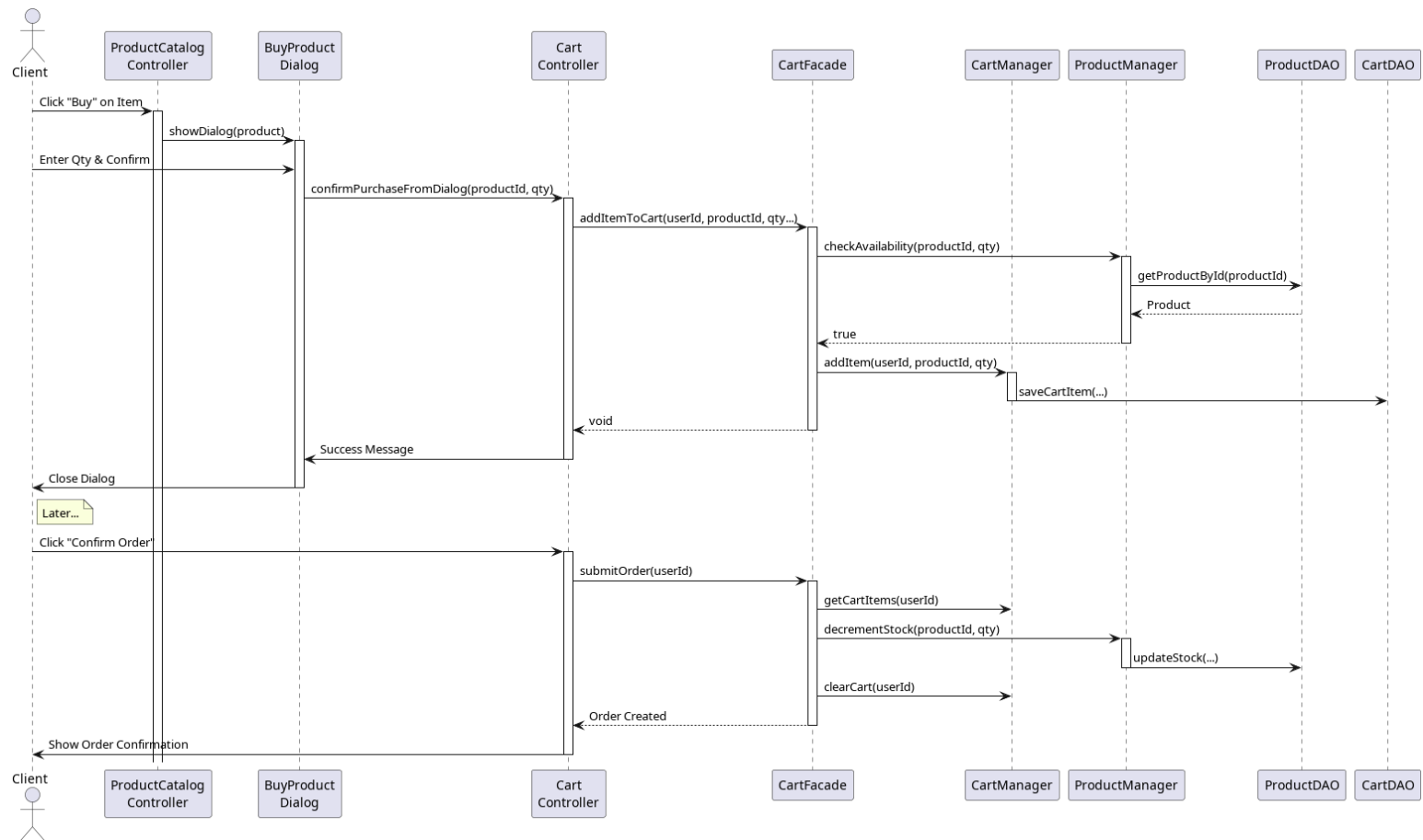




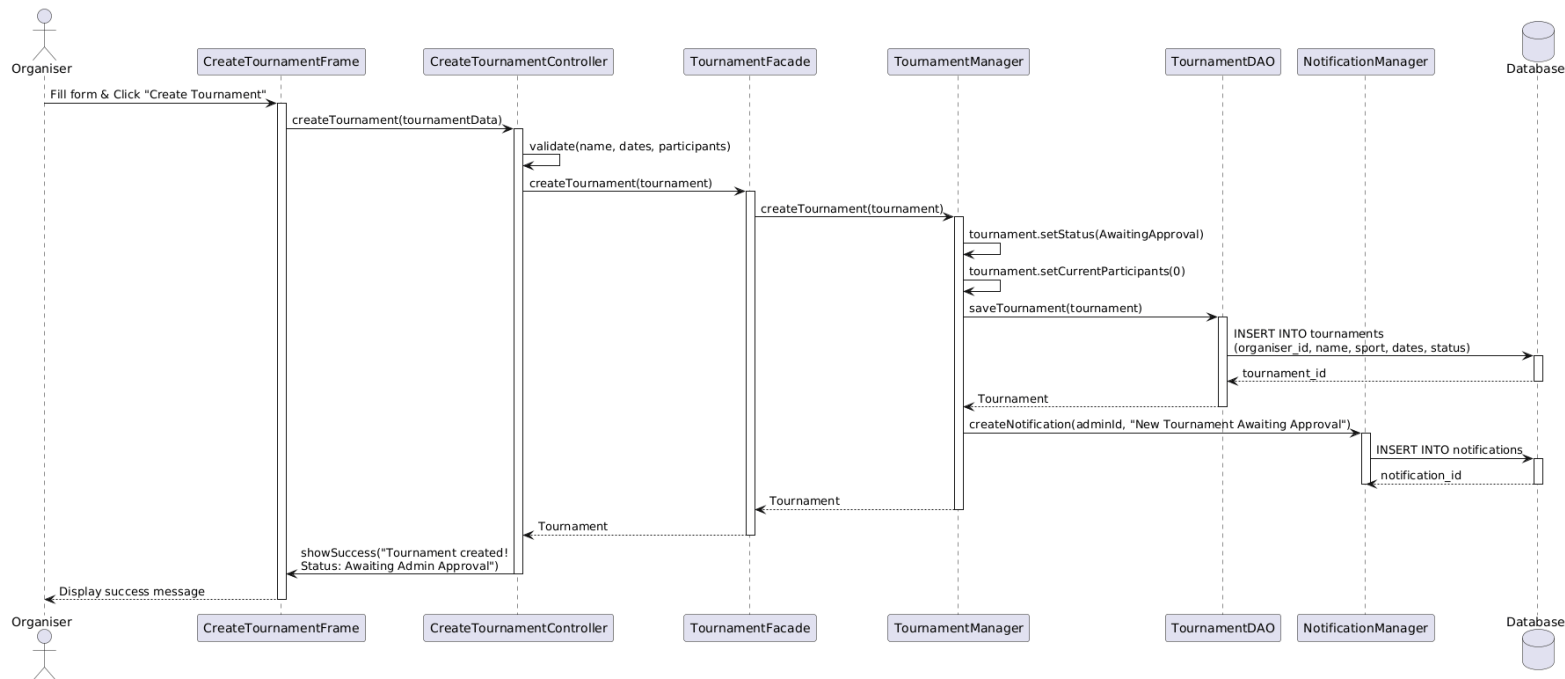
2. Notification Management:

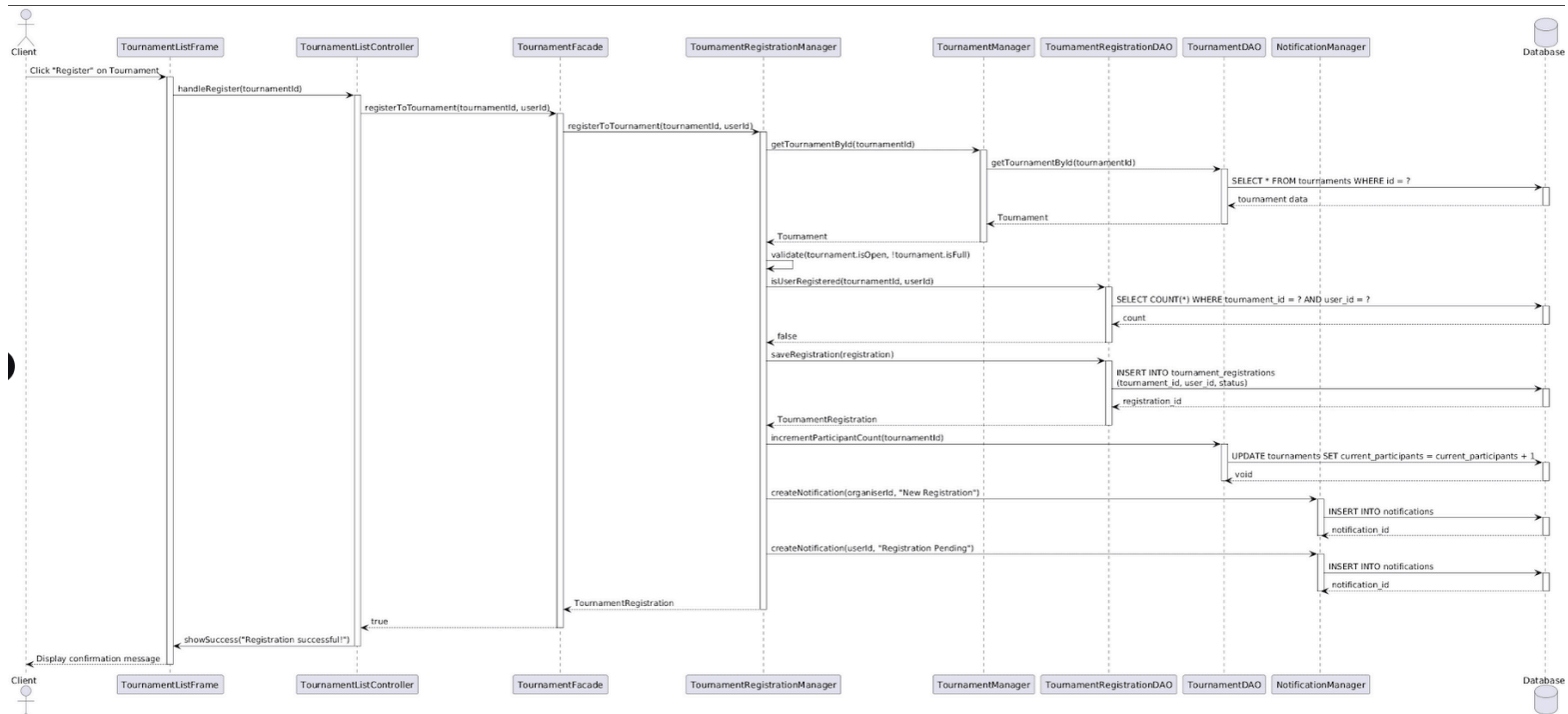


3. Equipment Management:



4. Tournament Management:





C. Second Sprint Management Report:

Date: Monday, January 5, 2026

Duration: 3 hours

Team Members: 4 people

Deadline: Monday, January 12, 2026

Manager: BEN AYED Ismaël

1. Project Overview

Our team is developing a Java application that serves as an intermediary platform connecting three main user groups: clients, tournament organizers, and sports facility owners. The platform allows clients to book sports facilities, participate in tournaments, and rent or buy sports equipment. Facility owners can manage their properties, define availability, and sell equipment. Tournament organizers can create and manage sporting events.

2. Objectives of the Session

According to the phase instructions, the team was required to:

1. Bring together all use cases from Sprint 2 (Authentication, Subscription Management, Terrain/Installation Management, and Reservation Management) to create a complete and working system.
2. Design UML class diagrams for the use cases that were assigned to each team member during the first sprint.
3. Develop the assigned use cases to ensure they work together properly and validate the proposed architecture.
4. Make sure that all designs followed the same architectural principles used in previous sprints.
5. Prepare the application for larger development by clearly defining the responsibilities and interfaces between modules.
6. Set up and use a shared remote database to support the team's development work.

3. Process Followed during the Session

3.1. Preparing the Environment

Before starting the design phase, the team agreed to reuse the same architectural template established during the previous sprint.

3.2. Class Diagram Preparation

Each team member was assigned two use cases.

During this sprint, each team member revisited the use cases assigned in the first sprint to create the design and begin the implementation.

UML class diagrams were created for the selected use cases, and sequence diagrams were produced when relevant.

The design work focused on:

- Proper separation of concerns
- Reusability of domain entities
- Consistency in naming conventions
- Alignment with Java and JavaFX constraints

3.3. Infrastructure Decisions

To avoid inconsistencies between local environments, the team decided to use a **remote PostgreSQL database hosted on Neon**.

This ensured that all members could work with the same database structure during development.

4. Work Distribution

Responsibilities were distributed based on functional domains:

- **Ismaël (Sprint Manager):**
 - Notifications (Design & Development)
- **Wassim:**
 - Tournament (Design & Development)
- **Syrine:**
 - Equipment Management (Design & Development)
 - Merging all developed use cases
- **Fatima:**
 - Cart Management (Design & Development)

5. Tools and Deliverables

5.1. UML Modeling Tool

- **StarUML:** For creating Class and Sequence diagrams.
- **JavaFX:** For the Graphical User Interface.
- **GitHub:** For version control and collaboration.
- **IntelliJ IDEA:** IDE used for Java development.
- **PostgreSQL:** For the database backend (with Neon).

5.2. Deliverables Produced

- **UML class diagrams for:**
 - Notifications
 - Tournament
 - Equipment Management
 - Cart Management
- UML sequence diagrams for selected use cases
- Implementation of the selected use cases
- Merging of all developed use cases into the main development branch
- Remote database configuration
- Updated design documentation

6. Process Metrics and Quantitative Overview

The design and implementation work for this sprint was conducted over two main working sessions of approximately 3 hours each. In addition, several team members continued working asynchronously during the holiday period to finalize implementation and integration tasks.

The UML design phase required approximately 2 hours and focused on the definition of class diagrams and sequence diagrams for the selected use cases. The implementation phase required more time, as it involved coding, testing, and integrating the designed use cases into the existing application architecture.

During this sprint:

- 4 major use cases were designed (Authentication, Subscription Management, Terrain/Installation Management, and Reservation Management)
- 4 use cases were partially or fully implemented

7. Problems Encountered and Lessons Learned

The main challenge during this sprint was assembling and merging the related use cases together. Since multiple members were working on interconnected modules, integrating them into a shared codebase required careful coordination, especially when core files were modified by different team members, leading to merge conflicts.

However, the design work was less complicated than at the beginning of the project because the team reproduced the same architecture across all use cases. This consistency made it easier to create UML diagrams and maintain a uniform structure throughout the system. The integration issues were resolved through improved communication and a more structured merging process, highlighting the importance of early synchronization and clear responsibility management.

8. Next Steps and Pending Items

The next sprint will be the final phase of the project. The team will write the management reports as well as the personal feedback on the project and test the application for the last time before delivering it.

9. Conclusion

This sprint successfully integrated all use cases from Sprint 1 into a working system. Each team member created designs and began implementing their assigned use cases using the established architectural patterns.

The main challenge was merging the interconnected modules, though design work was easier thanks to the consistent architecture applied across all use cases. With the foundation now in place, the team is ready for the final sprint to complete the remaining use cases and deliver a fully functional application.